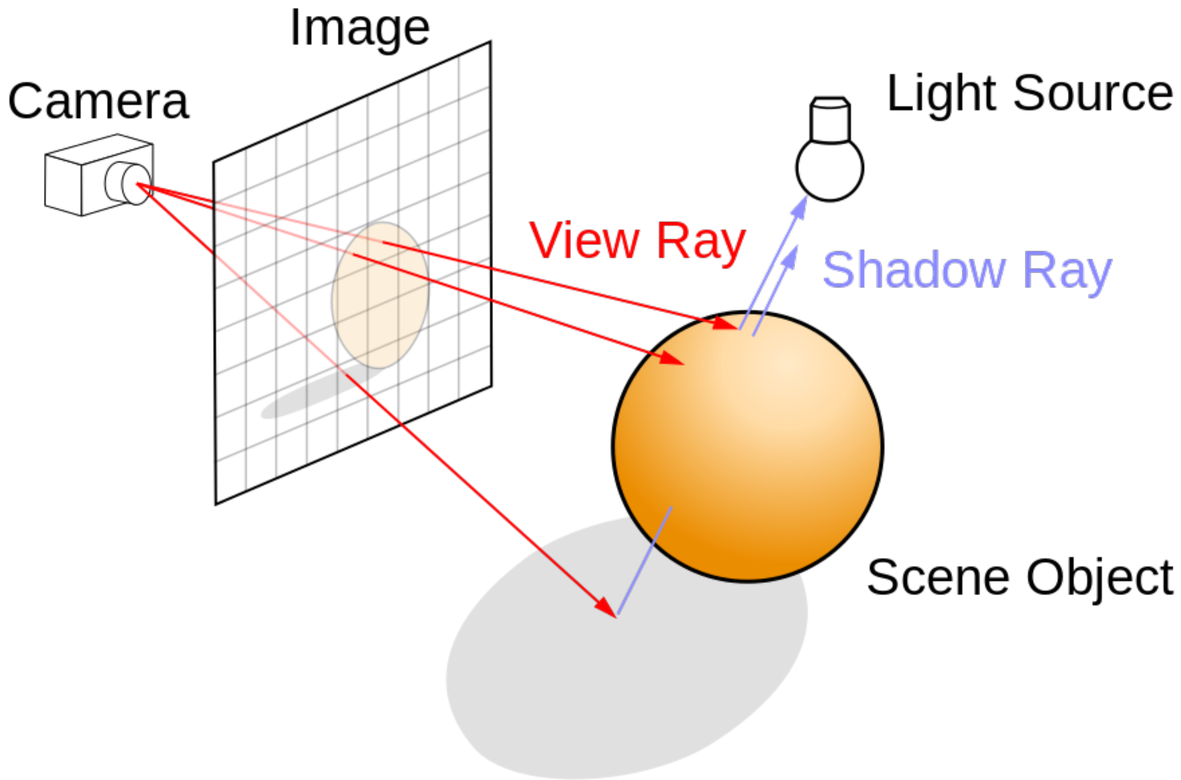
2D Transformations

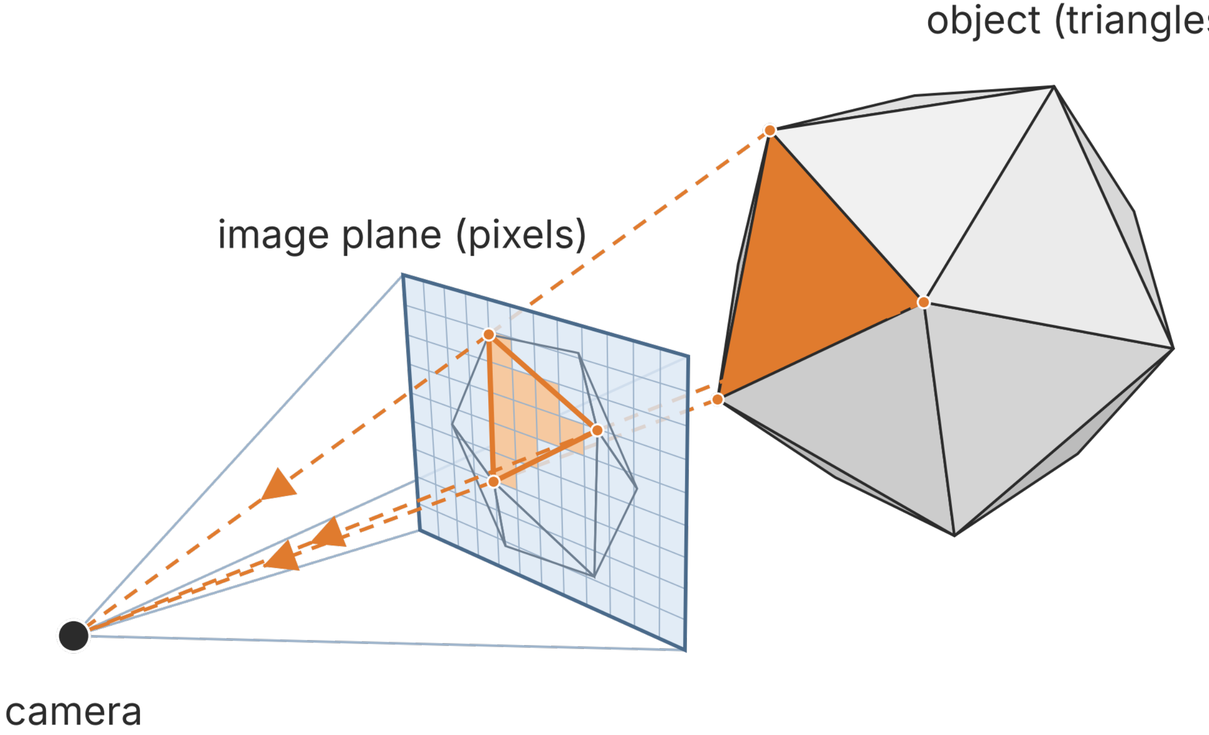
University of Victoria | Computer Science

Ray tracing → Rasterization

Per-pixel - "Raytracing"

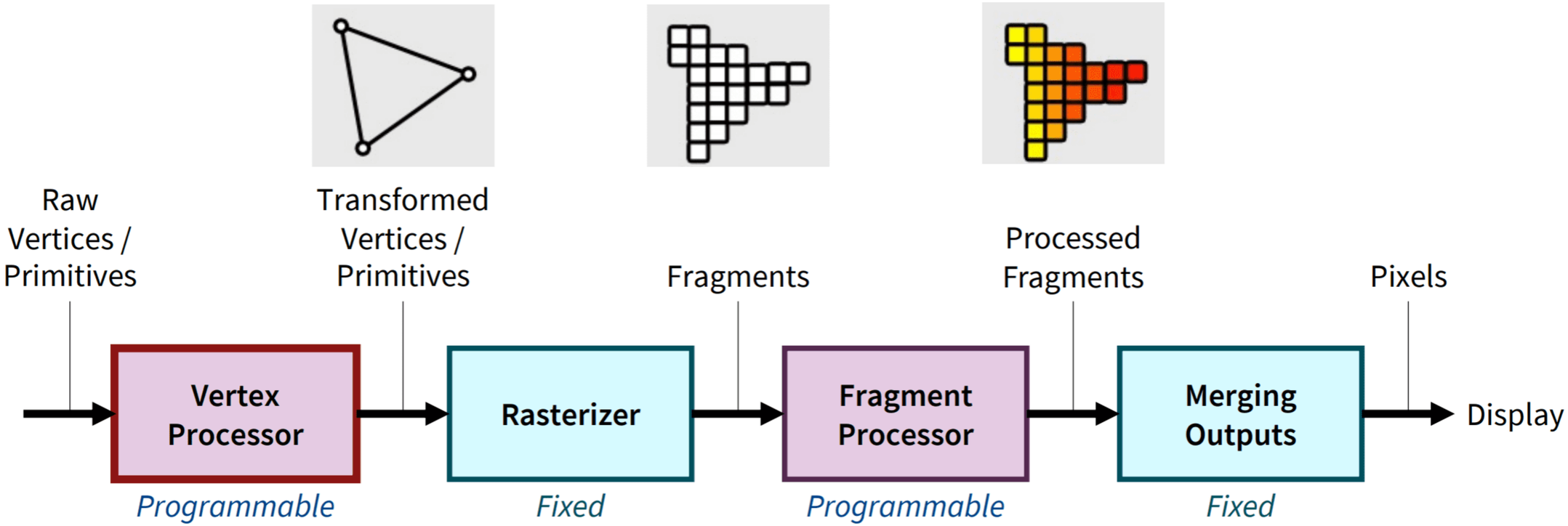


Per-object - "Rasterization"



University of Victoria | Computer Science

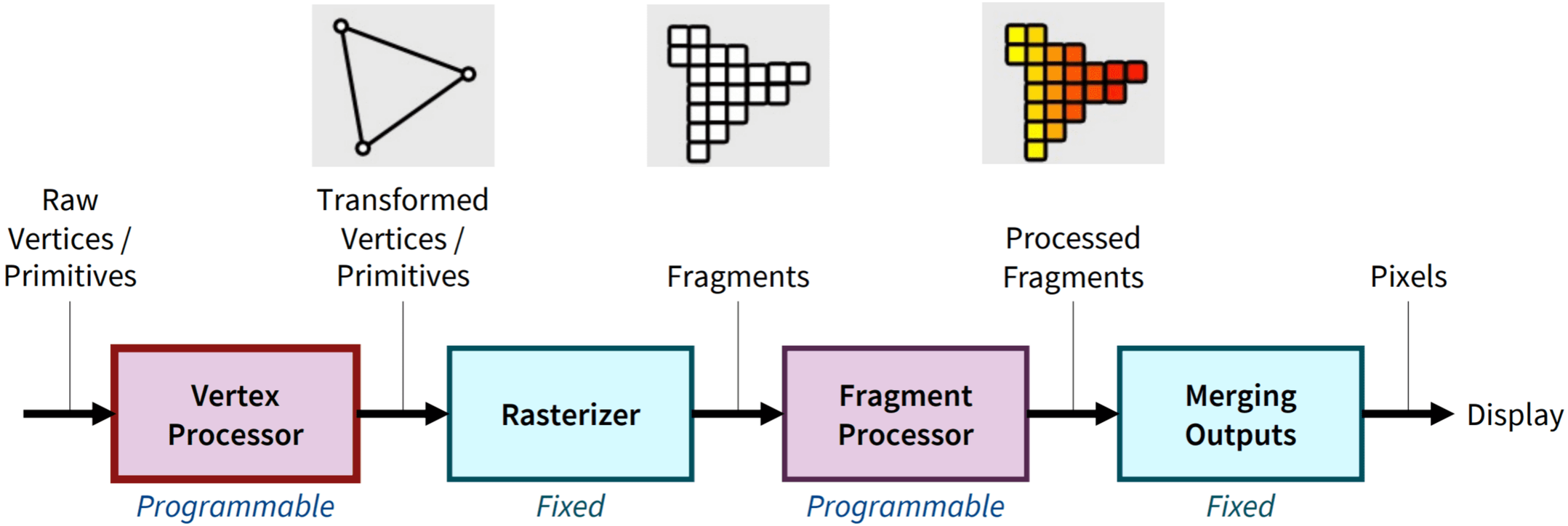
Ray tracing → Rasterization



Youngdo Lee; https://leeyngdo.github.io/blog/computer-graphics/2024-02-29-graphics-pipeline/

University of Victoria | Computer Science

Ray tracing → Rasterization



Youngdo Lee; https://leeyngdo.github.io/blog/computer-graphics/2024-02-29-graphics-pipeline/

University of Victoria | Computer Science

2D Linear Transformations

University of Victoria | Computer Science

2D Linear Transformations

- Each 2D linear map can be represented by a unique 2×2 matrix

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$$

University of Victoria | Computer Science

2D Linear Transformations

- Each 2D linear map can be represented by a unique 2×2 matrix

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$$

- Concatenation of mappings corresponds to multiplication of matrices

University of Victoria | Computer Science

2D Linear Transformations

- Each 2D linear map can be represented by a unique 2×2 matrix

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$$

- Concatenation of mappings corresponds to multiplication of matrices

$$L_2(L_1(\mathbf{x})) = \mathbf{L}_2 \mathbf{L}_1 \mathbf{x}$$

University of Victoria | Computer Science

2D Linear Transformations

$$L_2(L_1(\mathbf{x})) \;=\; \mathbf{L}_2\,\mathbf{L}_1\,\mathbf{x}$$

University of Victoria | Computer Science

2D Linear Transformations

$$L_2(L_1(\mathbf{x})) \;=\; \mathbf{L}_2\,\mathbf{L}_1\,\mathbf{x}$$

- Associative: brackets are free

$$L_2(L_1(\mathbf{x})) = \mathbf{L}_2(\mathbf{L}_1\mathbf{x}) = (\mathbf{L}_2\mathbf{L}_1)\,\mathbf{x}$$

University of Victoria | Computer Science

2D Linear Transformations

$$L_2(L_1(\mathbf{x})) \;=\; \mathbf{L}_2\,\mathbf{L}_1\,\mathbf{x}$$

- Associative: brackets are free

$$L_2(L_1(\mathbf{x})) = \mathbf{L}_2(\mathbf{L}_1\mathbf{x}) = (\mathbf{L}_2\mathbf{L}_1)\,\mathbf{x}$$

- Not commutative: order matters

$$\mathbf{L}_2\mathbf{L}_1 \neq \mathbf{L}_1\mathbf{L}_2 \quad (\text{in general})$$

University of Victoria | Computer Science

2D Linear Transformations

$$L_2(L_1(\mathbf{x})) \;=\; \mathbf{L}_2\,\mathbf{L}_1\,\mathbf{x}$$

- Associative: brackets are free

University of Victoria | Computer Science

2D Linear Transformations

$$L_2(L_1(\mathbf{x})) \;=\; \mathbf{L}_2\,\mathbf{L}_1\,\mathbf{x}$$

- Associative: brackets are free

Batch transform n points: $\mathbf{X} = \begin{bmatrix} | & | & & | \\ \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_n \\ | & | & & | \end{bmatrix} \quad (2 \times n)$

2D Linear Transformations

$$L_2(L_1(\mathbf{x})) \;=\; \mathbf{L}_2\,\mathbf{L}_1\,\mathbf{x}$$

- Associative: brackets are free

Batch transform n points: $\mathbf{X} = \begin{bmatrix} | & | & & | \\ \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_n \\ | & | & & | \end{bmatrix} \quad (2 \times n)$

Applying transformations one by one: $\mathbf{L}_3\big(\mathbf{L}_2(\mathbf{L}_1\mathbf{X})\big) : \quad 4n + 4n + 4n = 12n$

University of Victoria | Computer Science

2D Linear Transformations

$$L_2(L_1(\mathbf{x})) \;=\; \mathbf{L}_2\,\mathbf{L}_1\,\mathbf{x}$$

- Associative: brackets are free

Batch transform n points: $\mathbf{X} = \begin{bmatrix} | & | & & | \\ \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_n \\ | & | & & | \end{bmatrix} \quad (2 \times n)$

Applying transformations one by one: $\mathbf{L}_3\big(\mathbf{L}_2(\mathbf{L}_1\mathbf{X})\big)$: $\quad 4n + 4n + 4n = 12n$

Combine transformations first: $\underbrace{(\mathbf{L}_3\mathbf{L}_2\mathbf{L}_1)}_{\mathbf{M}:\ 8+8\ =\ 16}\ \mathbf{X}$: $\quad 16 + 4n$

University of Victoria | Computer Science

2D Linear Transformations

- Each 2D linear map can be represented by a unique 2×2 matrix

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$$

- Concatenation of mappings corresponds to multiplication of matrices

- Linear transformations are very common in computer graphics!

2D Linear Transformations

- Each 2D linear map can be represented by a unique 2×2 matrix

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$$

- Concatenation of mappings corresponds to multiplication of matrices

$$L_2(L_1(\mathbf{x})) = \mathbf{L}_2 \mathbf{L}_1 \mathbf{x}$$

- Linear transformations are very common in computer graphics!

University of Victoria | Computer Science

2D Linear Transformations

- Each 2D linear map can be represented by a unique 2×2 matrix

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$$

- Concatenation of mappings corresponds to multiplication of matrices

$$L_2(L_1(\mathbf{x})) = \mathbf{L}_2 \mathbf{L}_1 \mathbf{x}$$

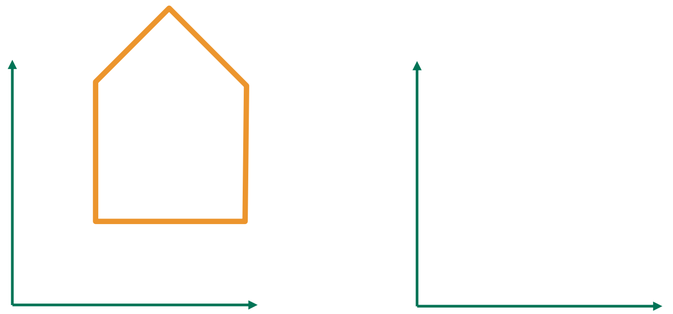
Eigen:
L2 * L1 * x;

- Linear transformations are very common in computer graphics!

University of Victoria | Computer Science

2D Scaling

- Scaling $\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}}_{\mathbf{S}(s_x, s_y)} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$



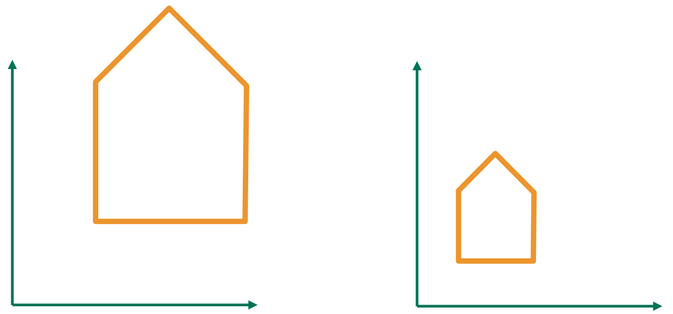
$\mathbf{S}(0.5, 0.5)$

$$x' = s_x \cdot x$$

$$y' = s_y \cdot y$$

2D Scaling

- Scaling $\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}}_{\mathbf{S}(s_x, s_y)} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$



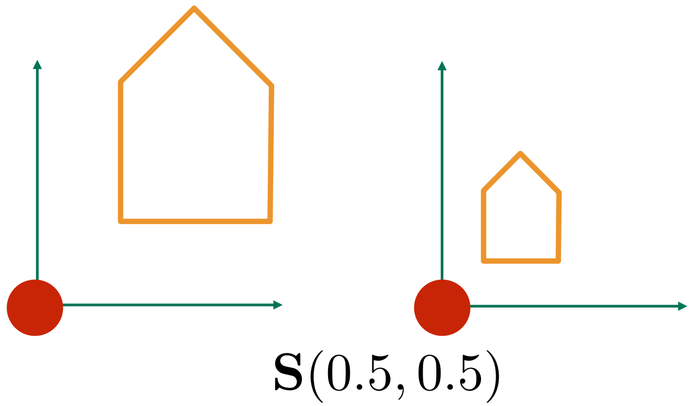
$\mathbf{S}(0.5, 0.5)$

$$x' = s_x \cdot x$$

$$y' = s_y \cdot y$$

2D Scaling

- Scaling $\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} s_x & 0 \\ 0 & s_y \end{pmatrix}}_{\mathbf{S}(s_x, s_y)} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$

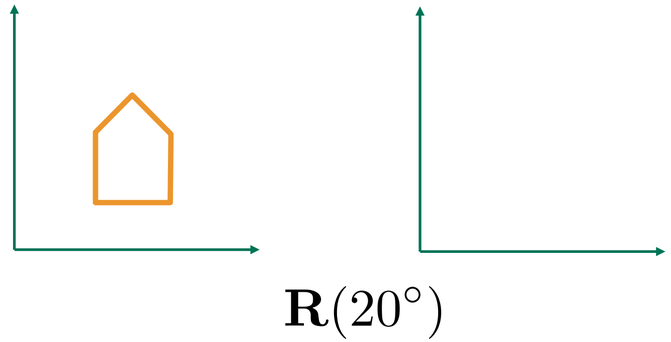


$$x' = s_x \cdot x$$

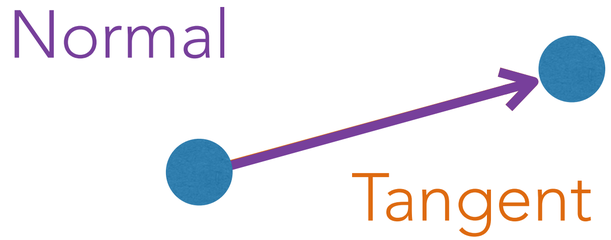
$$y' = s_y \cdot y$$

2D Rotation

- Rotation $\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{pmatrix}}_{\mathbf{R}(\alpha)} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$

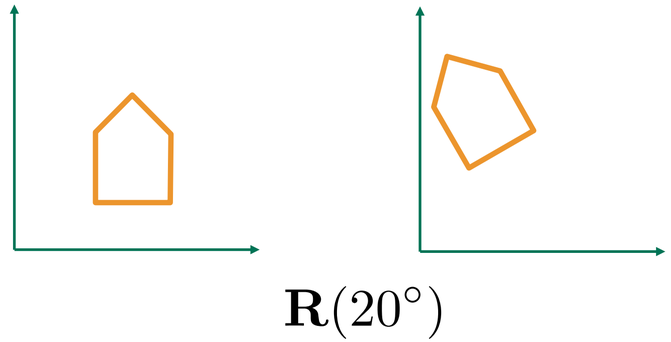


Special case: $\mathbf{R}(90) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$

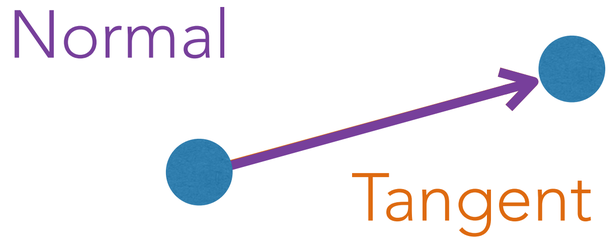


2D Rotation

- Rotation $\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{pmatrix}}_{\mathbf{R}(\alpha)} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$

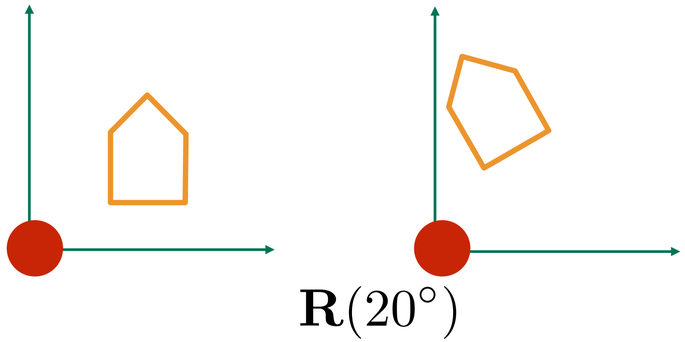


Special case: $\mathbf{R}(90) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$

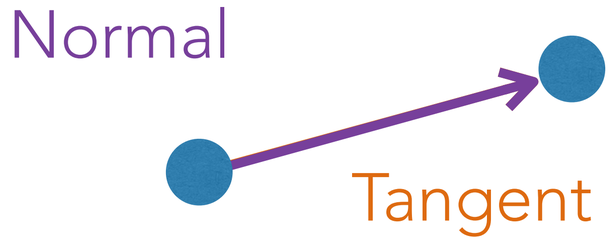


2D Rotation

- Rotation $\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{pmatrix}}_{\mathbf{R}(\alpha)} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$

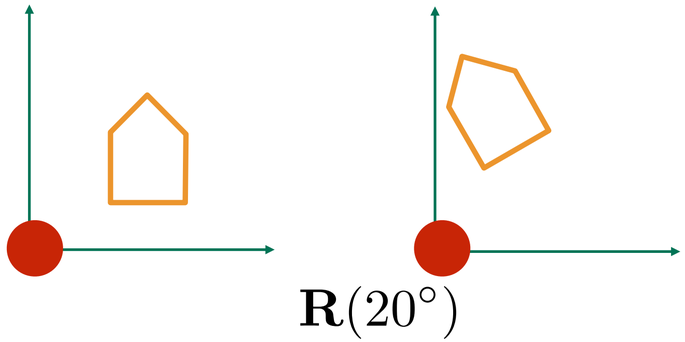


Special case: $\mathbf{R}(90) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$



2D Rotation

- Rotation $\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{pmatrix}}_{\mathbf{R}(\alpha)} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$



Special case: $\mathbf{R}(90) = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$

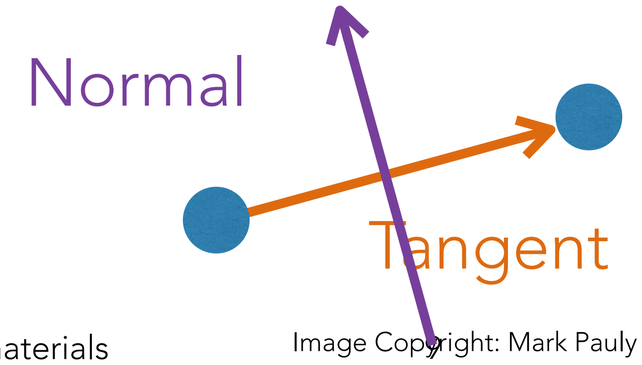
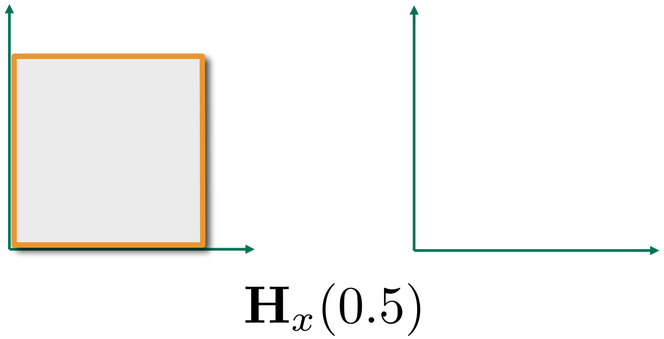


Image Copyright: Mark Pauly

2D Shearing

- Shear along x-axis

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & a \\ 0 & 1 \end{pmatrix}}_{\mathbf{H}_x(a)} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$$



- Shear along y-axis

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 0 \\ b & 1 \end{pmatrix}}_{\mathbf{H}_y(b)} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$$

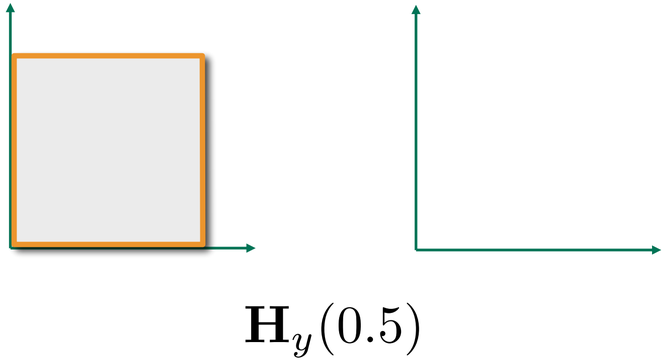
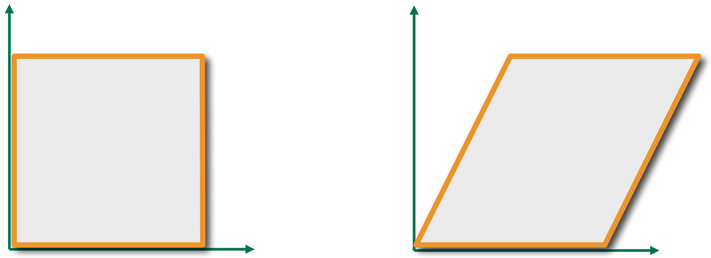


Image Copyright: Mark Pauly

2D Shearing

- Shear along x-axis

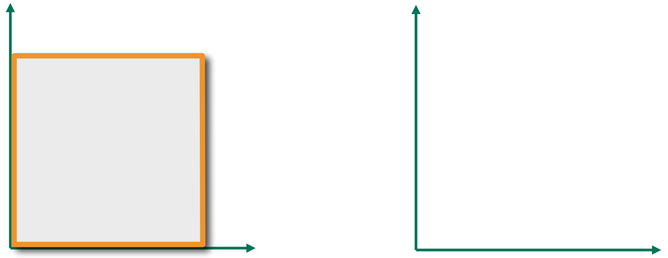
$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & a \\ 0 & 1 \end{pmatrix}}_{\mathbf{H}_x(a)} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$$



$\mathbf{H}_x(0.5)$

- Shear along y-axis

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 0 \\ b & 1 \end{pmatrix}}_{\mathbf{H}_y(b)} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$$

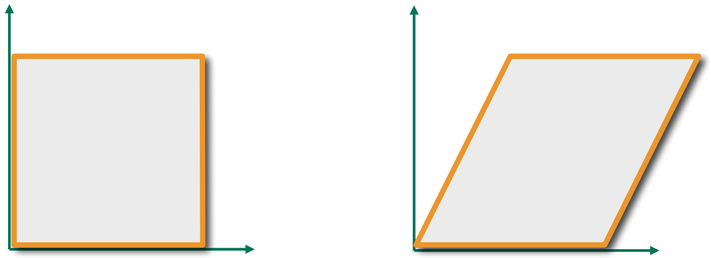


$\mathbf{H}_y(0.5)$

2D Shearing

- Shear along x-axis

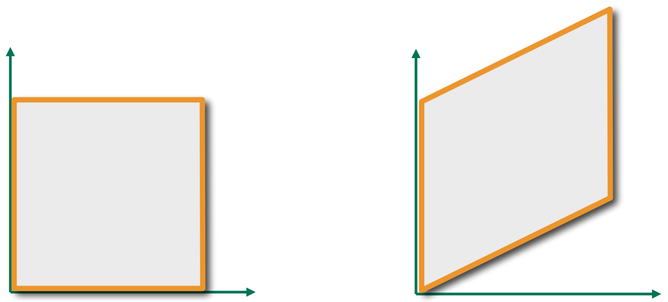
$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & a \\ 0 & 1 \end{pmatrix}}_{\mathbf{H}_x(a)} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$$



$\mathbf{H}_x(0.5)$

- Shear along y-axis

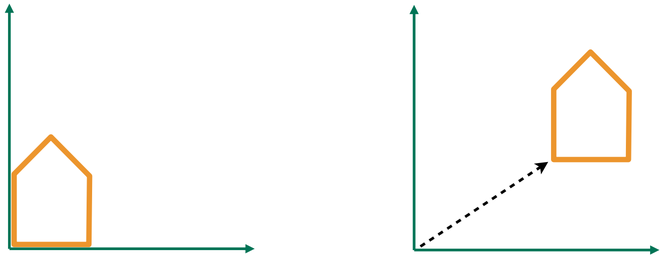
$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \underbrace{\begin{pmatrix} 1 & 0 \\ b & 1 \end{pmatrix}}_{\mathbf{H}_y(b)} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$$



$\mathbf{H}_y(0.5)$

2D Translation

- Translation $\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix}$



- Matrix representation? $\begin{pmatrix} x' \\ y' \end{pmatrix} = \mathbf{T}(t_x, t_y) \cdot \begin{pmatrix} x \\ y \end{pmatrix}$

Affine Transformations

Affine Transformations

- Translation is not linear, but it is *affine*
  - Origin is no longer a fixed point

University of Victoria | Computer Science

Affine Transformations

- Translation is not linear, but it is *affine*
  - Origin is no longer a fixed point
- Affine map = linear map + translation

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix} = \mathbf{Lx} + \mathbf{t}$$

University of Victoria | Computer Science

Affine Transformations

- Translation is not linear, but it is *affine*
  - Origin is no longer a fixed point
- Affine map = linear map + translation

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix} = \mathbf{Lx} + \mathbf{t}$$

- Is there a matrix representation for affine transformations?
  - We would like to handle all transformations in a unified framework -> simpler to code and easier to optimize!

Homogeneous Coordinates

University of Victoria | Computer Science

Homogeneous Coordinates

- Add a third coordinate (*w-coordinate*)

University of Victoria | Computer Science

Homogeneous Coordinates

- Add a third coordinate (*w-coordinate*)
  - 2D point = $(x, y, 1)^T$

University of Victoria | Computer Science

Homogeneous Coordinates

- Add a third coordinate (*w-coordinate*)
  - 2D point = $(x, y, 1)^T$
  - 2D vector = $(x, y, 0)^T$

University of Victoria | Computer Science

Homogeneous Coordinates

- Add a third coordinate (*w-coordinate*)
  - 2D point = (x, y, 1)^T
  - 2D vector = (x, y, 0)^T

$$\begin{pmatrix} x' \\ y' \\ w' \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ 1 \end{pmatrix}$$

- Matrix representation of translations

University of Victoria | Computer Science

Homogeneous Coordinates

University of Victoria | Computer Science

Homogeneous Coordinates

- Valid operation if the resulting *w*-coordinate is 1 or 0

University of Victoria | Computer Science

Homogeneous Coordinates

- Valid operation if the resulting *w*-coordinate is 1 or 0
  - vector + vector= vector $(x_1, y_1, 0) + (x_2, y_2, 0) = (x_1 + x_2,\ y_1 + y_2,\ 0)$

University of Victoria | Computer Science

Homogeneous Coordinates

- Valid operation if the resulting *w*-coordinate is 1 or 0
  - vector + vector= vector $(x_1, y_1, 0) + (x_2, y_2, 0) = (x_1 + x_2,\ y_1 + y_2,\ 0)$
  - point - point = vector $(x_1, y_1, 1) - (x_2, y_2, 1) = (x_1 - x_2,\ y_1 - y_2,\ 0)$

University of Victoria | Computer Science

Homogeneous Coordinates

- Valid operation if the resulting *w*-coordinate is 1 or 0
  - vector + vector = vector $(x_1, y_1, 0) + (x_2, y_2, 0) = (x_1 + x_2,\ y_1 + y_2,\ 0)$
  - point - point = vector $(x_1, y_1, 1) - (x_2, y_2, 1) = (x_1 - x_2,\ y_1 - y_2,\ 0)$
  - point + vector = point $(x_1, y_1, 1) + (x_2, y_2, 0) = (x_1 + x_2,\ y_1 + y_2,\ 1)$

University of Victoria | Computer Science

Homogeneous Coordinates

- Valid operation if the resulting *w*-coordinate is 1 or 0
  - vector + vector = vector $(x_1, y_1, 0) + (x_2, y_2, 0) = (x_1 + x_2,\ y_1 + y_2,\ 0)$
  - point - point = vector $(x_1, y_1, 1) - (x_2, y_2, 1) = (x_1 - x_2,\ y_1 - y_2,\ 0)$
  - point + vector = point $(x_1, y_1, 1) + (x_2, y_2, 0) = (x_1 + x_2,\ y_1 + y_2,\ 1)$
  - point + point = ??? $(x_1, y_1, 1) + (x_2, y_2, 1) = (x_1 + x_2,\ y_1 + y_2,\ 2)$

University of Victoria | Computer Science

Homogeneous Coordinates

- Valid operation if the resulting *w*-coordinate is 1 or 0
  - vector + vector = vector $(x_1, y_1, 0) + (x_2, y_2, 0) = (x_1 + x_2,\ y_1 + y_2,\ 0)$
  - point - point = vector $(x_1, y_1, 1) - (x_2, y_2, 1) = (x_1 - x_2,\ y_1 - y_2,\ 0)$
  - point + vector = point $(x_1, y_1, 1) + (x_2, y_2, 0) = (x_1 + x_2,\ y_1 + y_2,\ 1)$
  - point + point = ??? $(x_1, y_1, 1) + (x_2, y_2, 1) = (x_1 + x_2,\ y_1 + y_2,\ 2)$ ✗

Homogeneous Coordinates

- Geometric interpretation: 2 hyperplanes in $\mathbf{R}^3$

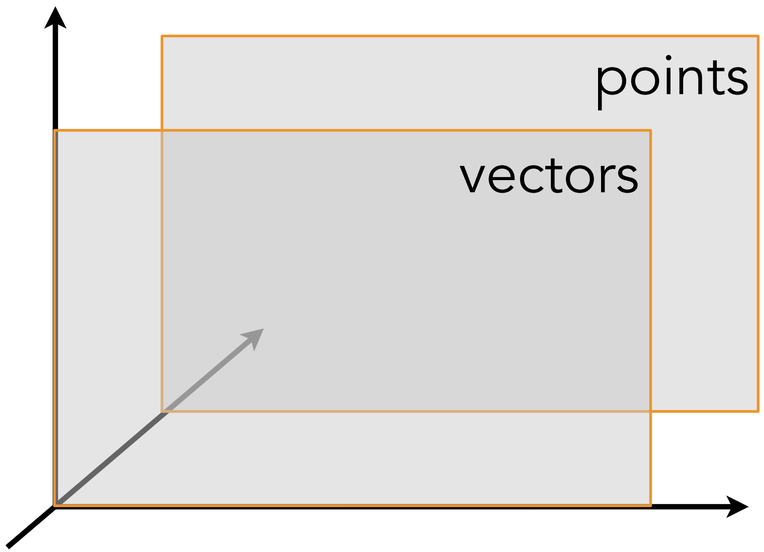


Image Copyright: Mark Pauly

Affine Transformations

- Affine map = linear map + translation

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix}$$

- Using homogenous coordinates:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Affine Transformations

- Affine map = linear map + translation

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix}$$

- Using homogenous coordinates:

Linear map

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

University of Victoria | Computer Science

Affine Transformations

- Affine map = linear map + translation

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix}$$

- Using homogenous coordinates:

Linear map Translation

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

University of Victoria | Computer Science

Affine Transformations

- Affine map = linear map + translation

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix}$$

- Using homogenous coordinates:

Linear map Translation

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Fixed to (0, 0, 1)

University of Victoria | Computer Science

2D Transformations

- Scale

$$\mathbf{S}(s_x, s_y) = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- Rotation

$$\mathbf{R}(\alpha) = \begin{pmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- Translation

$$\mathbf{T}(t_x, t_y) = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

University of Victoria | Computer Science

Concatenation of Transformations

- Sequence of affine maps $\mathbf{A}_1$, $\mathbf{A}_2$, $\mathbf{A}_3$, ...

University of Victoria | Computer Science

Concatenation of Transformations

- Sequence of affine maps $\mathbf{A}_1$, $\mathbf{A}_2$, $\mathbf{A}_3$, ...
  - Concatenation by matrix multiplication

University of Victoria | Computer Science

Concatenation of Transformations

- Sequence of affine maps $\mathbf{A}_1$, $\mathbf{A}_2$, $\mathbf{A}_3$, ...
  - Concatenation by matrix multiplication

$$A_n(\dots A_2(A_1(\mathbf{x}))) \;=\; \mathbf{A}_n \cdots \mathbf{A}_2 \cdot \mathbf{A}_1 \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

University of Victoria | Computer Science

Concatenation of Transformations

- Sequence of affine maps $\mathbf{A}_1$, $\mathbf{A}_2$, $\mathbf{A}_3$, ...
  - Concatenation by matrix multiplication

$$A_n(\dots A_2(A_1(\mathbf{x}))) \;=\; \mathbf{A}_n \cdots \mathbf{A}_2 \cdot \mathbf{A}_1 \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

  - Very important for performance!

University of Victoria | Computer Science

Concatenation of Transformations

- Sequence of affine maps $\mathbf{A}_1$, $\mathbf{A}_2$, $\mathbf{A}_3$, ...
  - Concatenation by matrix multiplication

$$A_n(\dots A_2(A_1(\mathbf{x}))) \;=\; \mathbf{A}_n \cdots \mathbf{A}_2 \cdot \mathbf{A}_1 \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

  - Very important for performance!
  - Matrix multiplication not commutative, ordering is important!

University of Victoria | Computer Science

Rotation and Translation

- Matrix multiplication is not commutative!

Image Copyright: Mark Pauly

Rotation and Translation

- Matrix multiplication is not commutative!
  - First rotation, then translation

Image Copyright: Mark Pauly

Rotation and Translation

- Matrix multiplication is not commutative!
  - First rotation, then translation

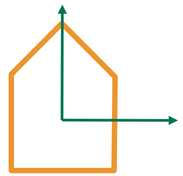


Image Copyright: Mark Pauly

Rotation and Translation

- Matrix multiplication is not commutative!
  - First rotation, then translation

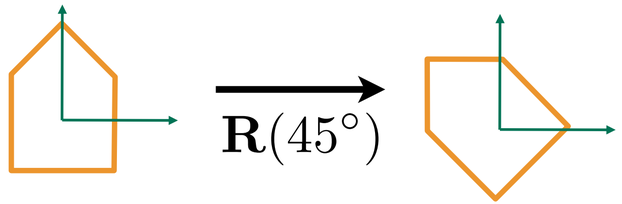


Image Copyright: Mark Pauly

Rotation and Translation

- Matrix multiplication is not commutative!
  - First rotation, then translation

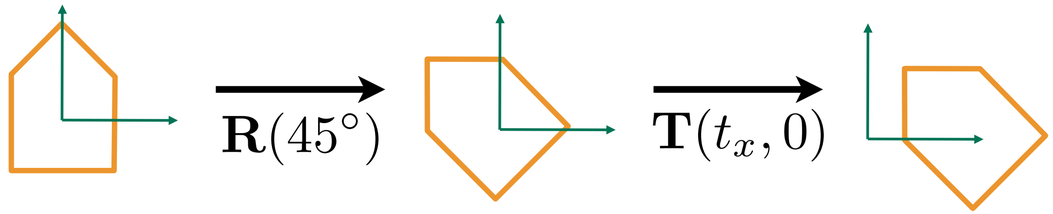
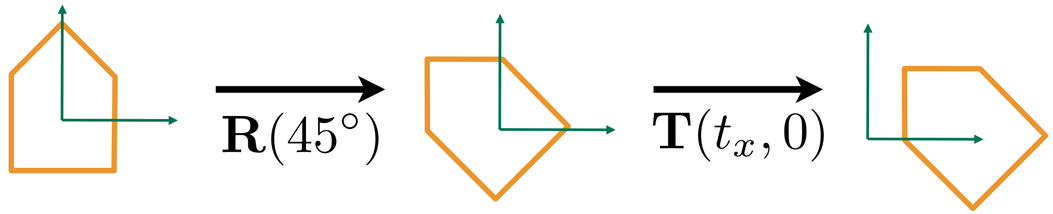


Image Copyright: Mark Pauly

Rotation and Translation

- Matrix multiplication is not commutative!
  - First rotation, then translation

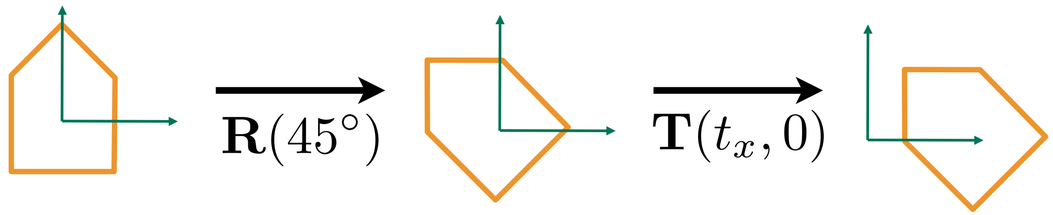


  - First translation, then rotation

Image Copyright: Mark Pauly

Rotation and Translation

- Matrix multiplication is not commutative!
  - First rotation, then translation



  - First translation, then rotation

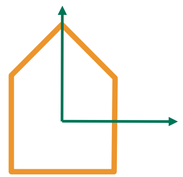
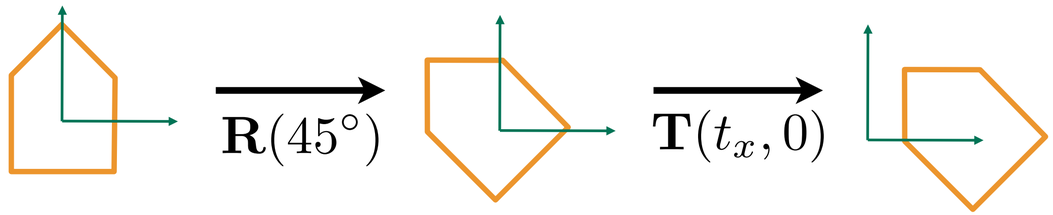


Image Copyright: Mark Pauly

Rotation and Translation

- Matrix multiplication is not commutative!
  - First rotation, then translation



  - First translation, then rotation

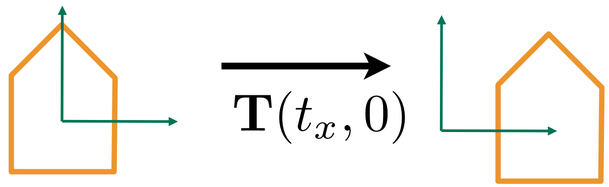
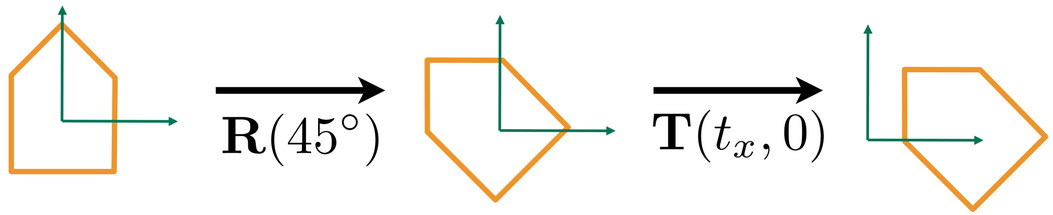


Image Copyright: Mark Pauly

Rotation and Translation

- Matrix multiplication is not commutative!
  - First rotation, then translation



  - First translation, then rotation

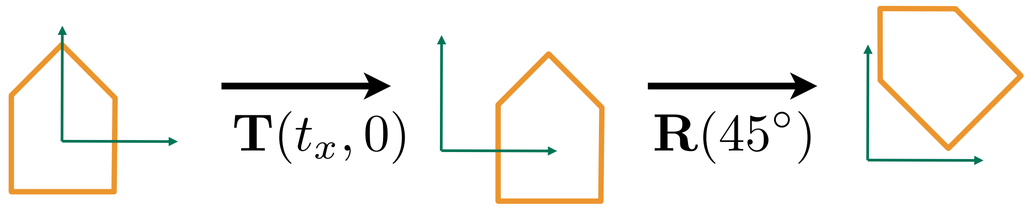
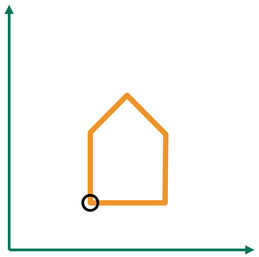


Image Copyright: Mark Pauly

2D Rotation

- How to rotate around a given point **c**?



2D Rotation

- How to rotate around a given point **c**?
  1. Translate **c** to origin

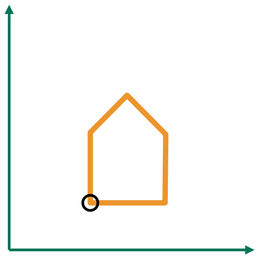
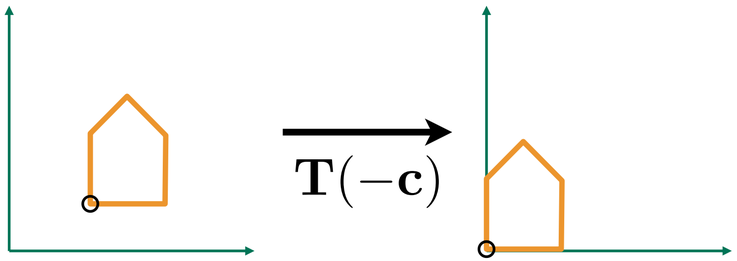


Image Copyright: Mark Pauly

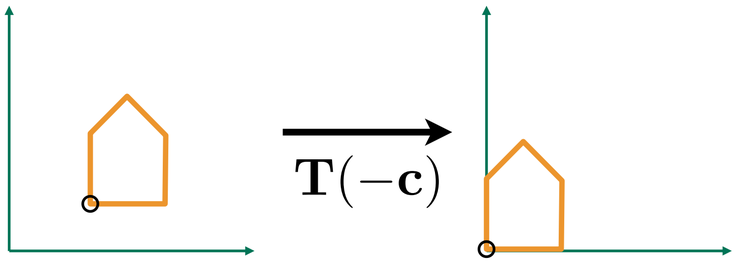
2D Rotation

- How to rotate around a given point **c**?
  1. Translate **c** to origin



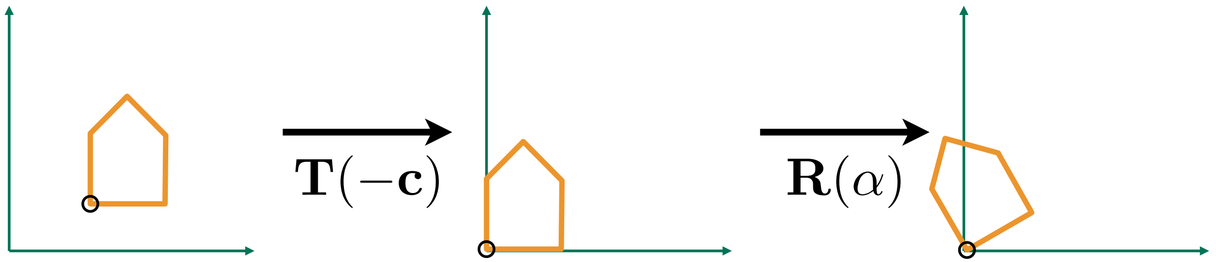
2D Rotation

- How to rotate around a given point **c**?
  1. Translate **c** to origin
  2. Rotate



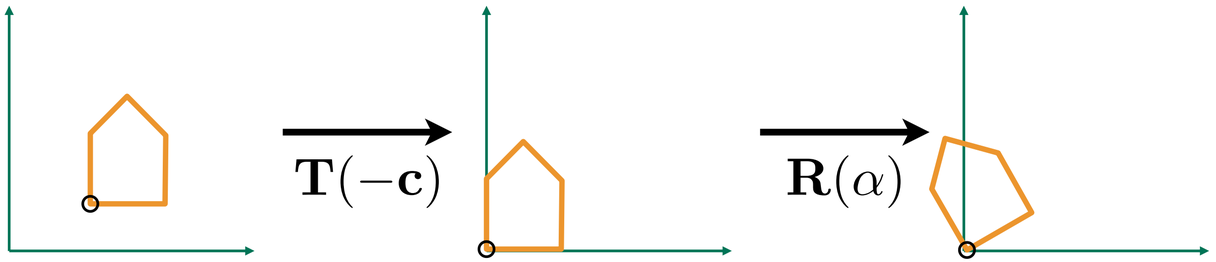
2D Rotation

- How to rotate around a given point **c**?
  1. Translate **c** to origin
  2. Rotate



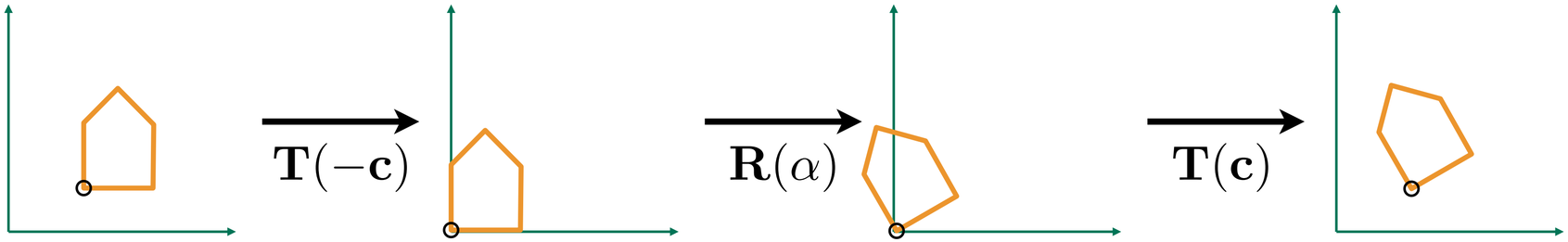
2D Rotation

- How to rotate around a given point **c**?
  1. Translate **c** to origin
  2. Rotate
  3. Translate back



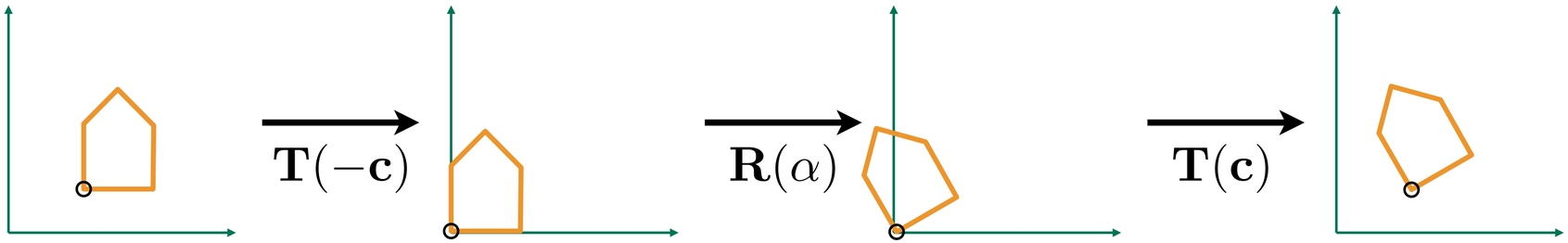
2D Rotation

- How to rotate around a given point **c**?
  1. Translate **c** to origin
  2. Rotate
  3. Translate back



2D Rotation

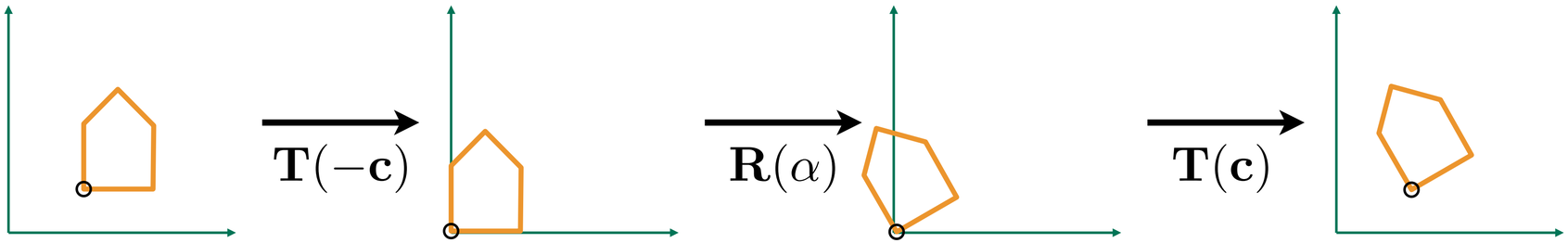
- How to rotate around a given point **c**?
  1. Translate **c** to origin
  2. Rotate
  3. Translate back



- Matrix representation?

2D Rotation

- How to rotate around a given point **c**?
  1. Translate **c** to origin
  2. Rotate
  3. Translate back

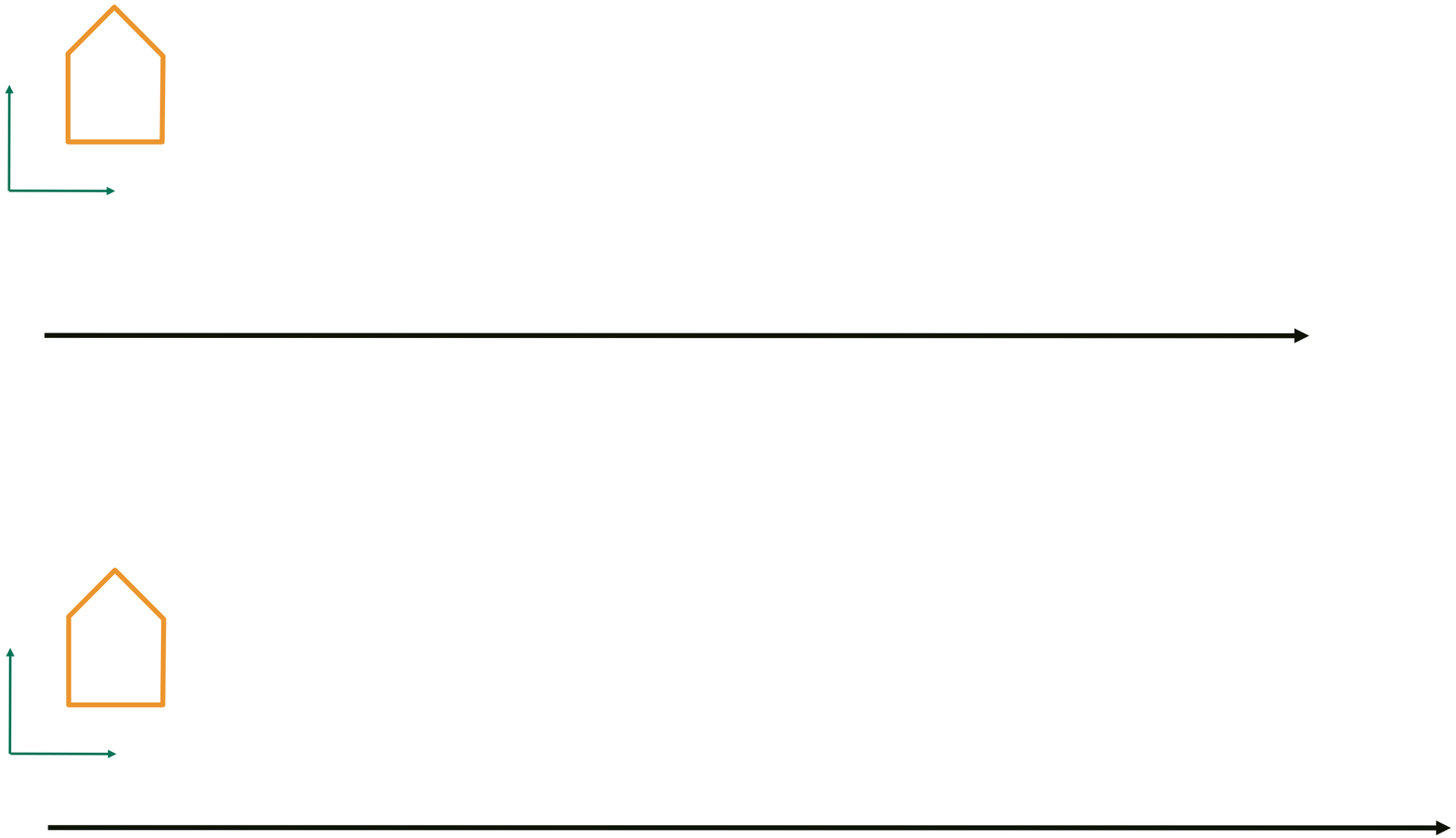


- Matrix representation?

$$\mathbf{T}(\mathbf{c}) \cdot \mathbf{R}(\alpha) \cdot \mathbf{T}(-\mathbf{c})$$

Image Copyright: Mark Pauly

Transform Object or Camera?



Transform Object or Camera?

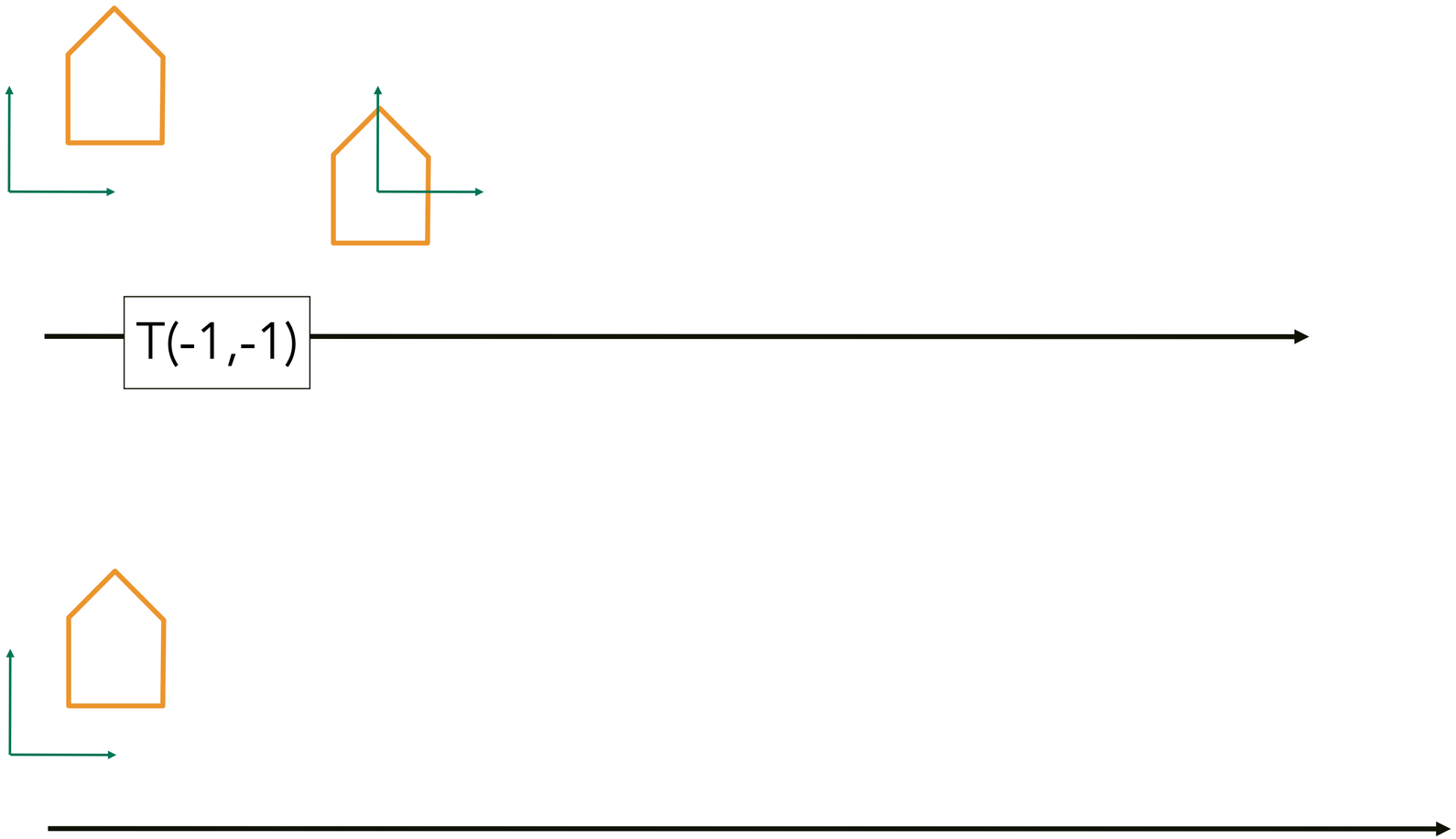


Image Copyright: Mark Pauly

Transform Object or Camera?

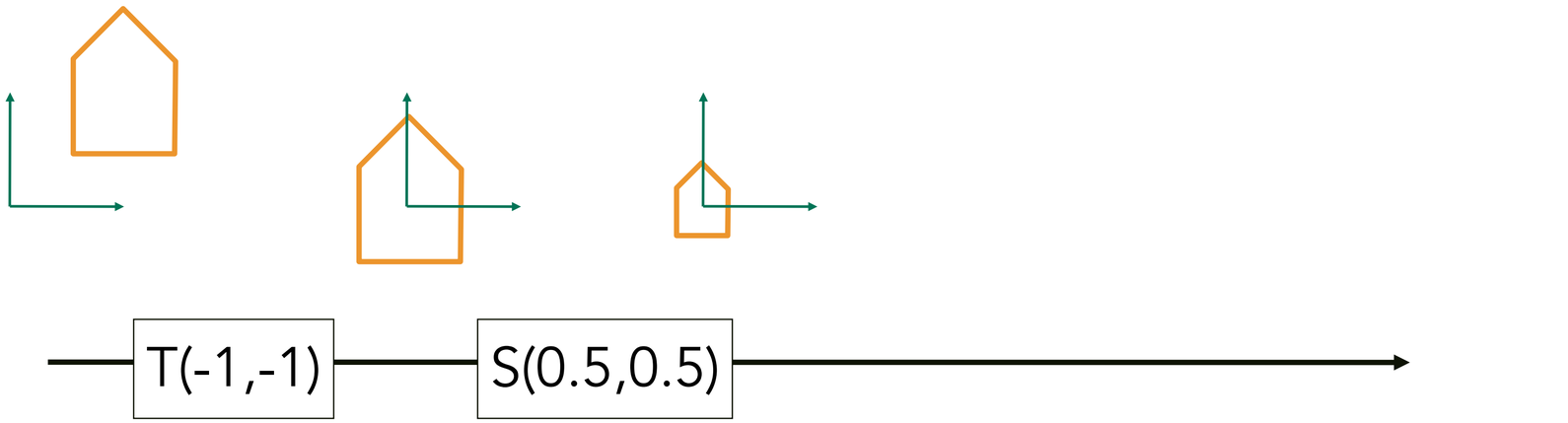




Image Copyright: Mark Pauly

Transform Object or Camera?

$$\mathbf{M} = \mathbf{T}(1,1)\,\mathbf{R}(45)\,\mathbf{S}(0.5,0.5)\,\mathbf{T}(-1,-1)$$

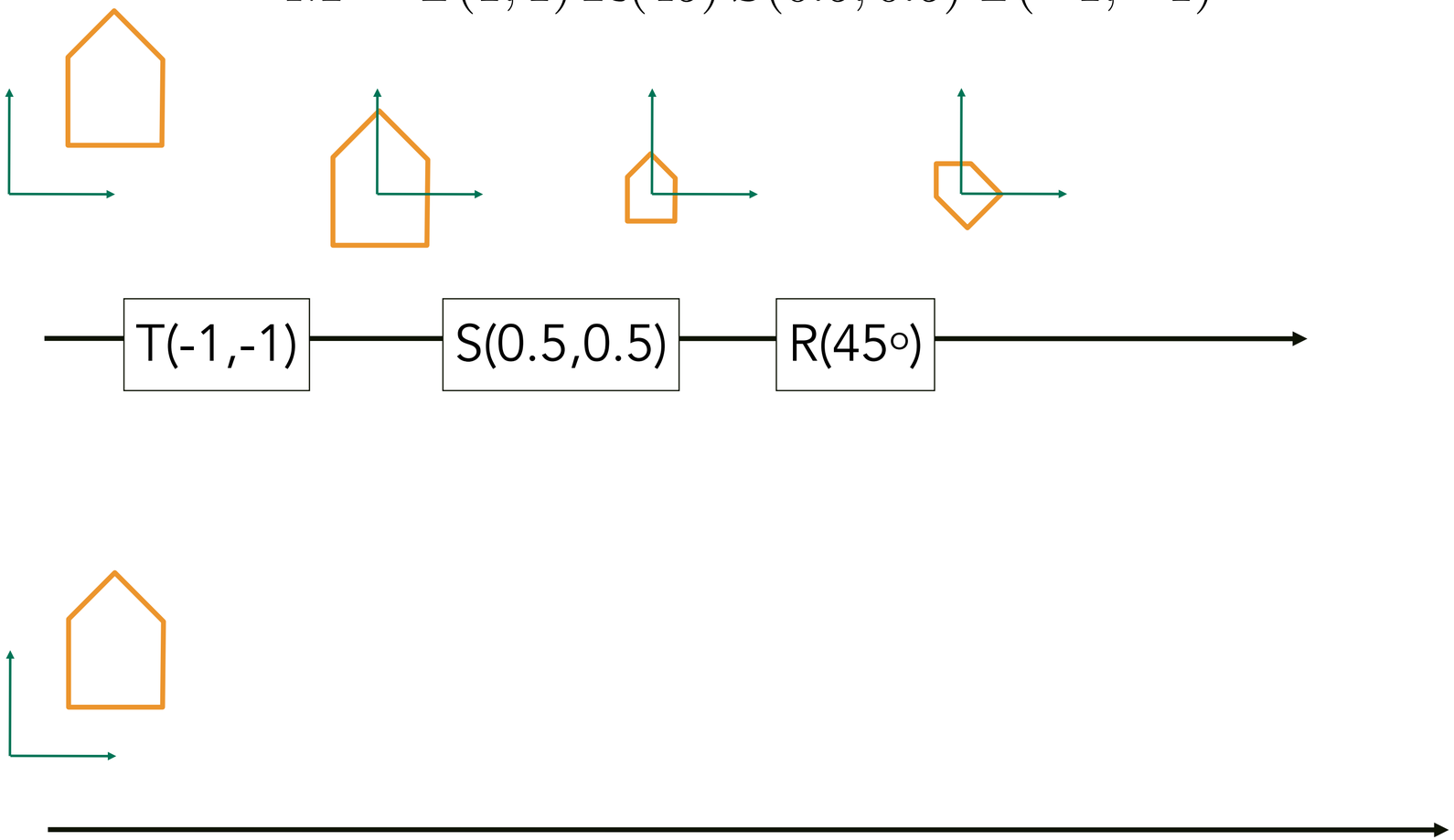


Image Copyright: Mark Pauly

Transform Object or Camera?

$$\mathbf{M} = \mathbf{T}(1,1)\,\mathbf{R}(45)\,\mathbf{S}(0.5,0.5)\,\mathbf{T}(-1,-1)$$

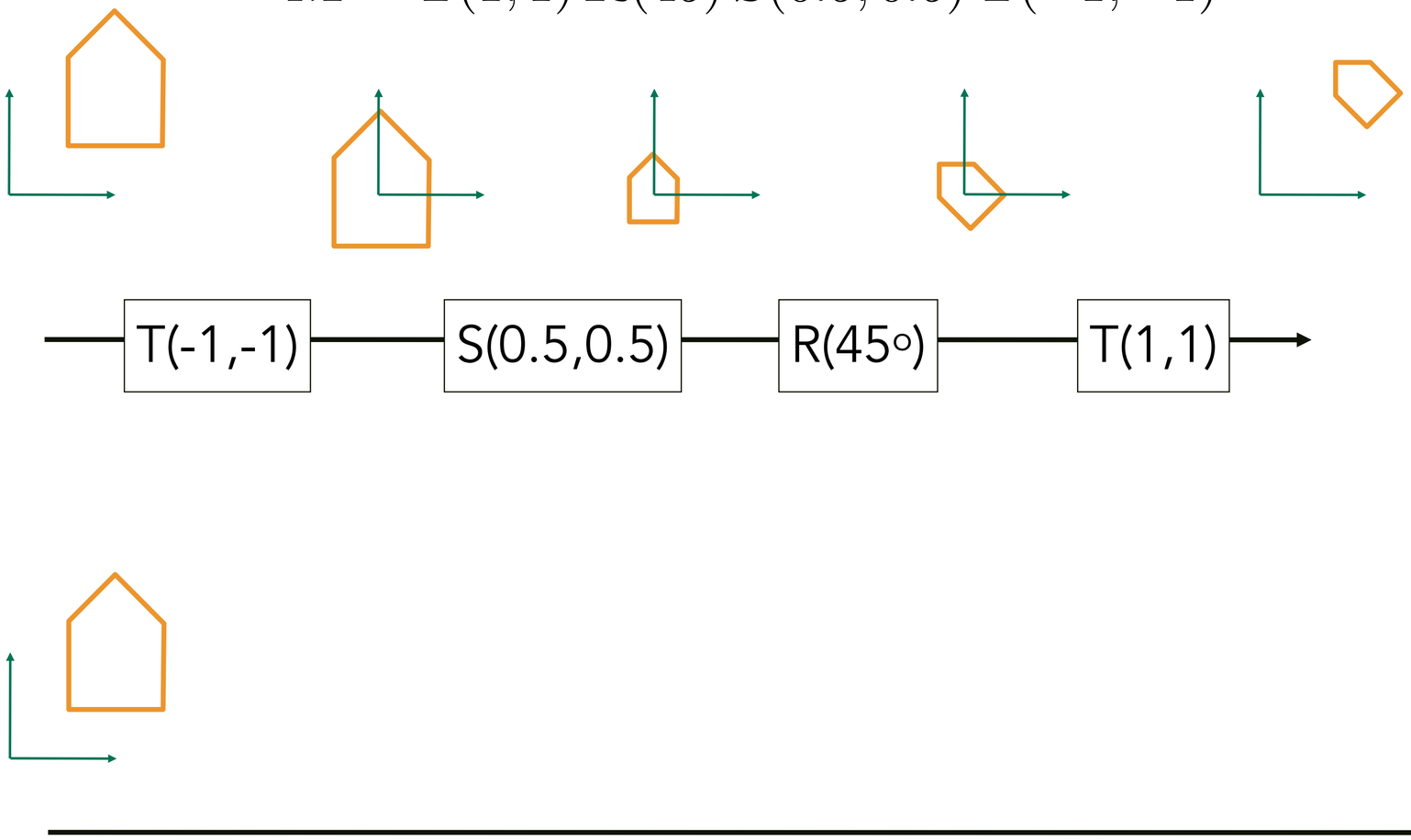


Image Copyright: Mark Pauly

Transform Object or Camera?

$$\mathbf{M} = \mathbf{T}(1,1)\,\mathbf{R}(45)\,\mathbf{S}(0.5,0.5)\,\mathbf{T}(-1,-1)$$

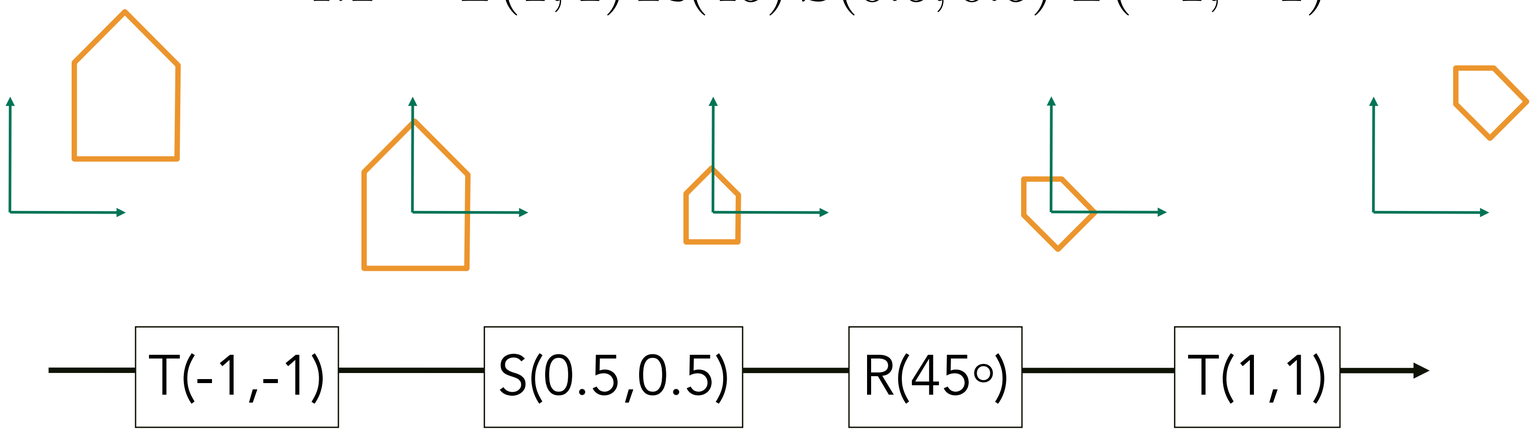
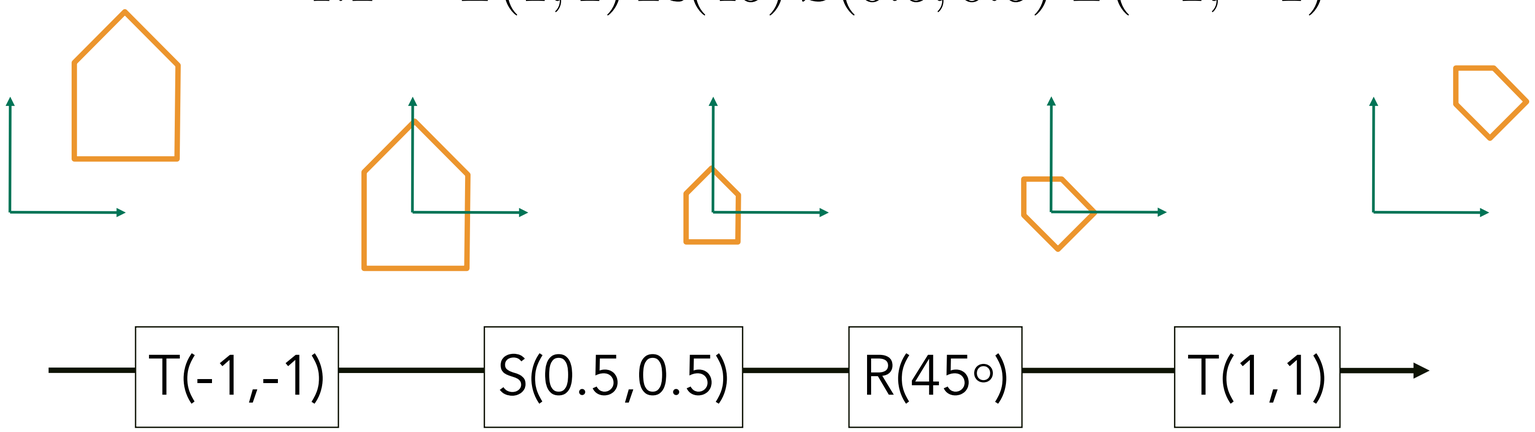




Image Copyright: Mark Pauly

Transform Object or Camera?

$$\mathbf{M} = \mathbf{T}(1,1)\,\mathbf{R}(45)\,\mathbf{S}(0.5,0.5)\,\mathbf{T}(-1,-1)$$



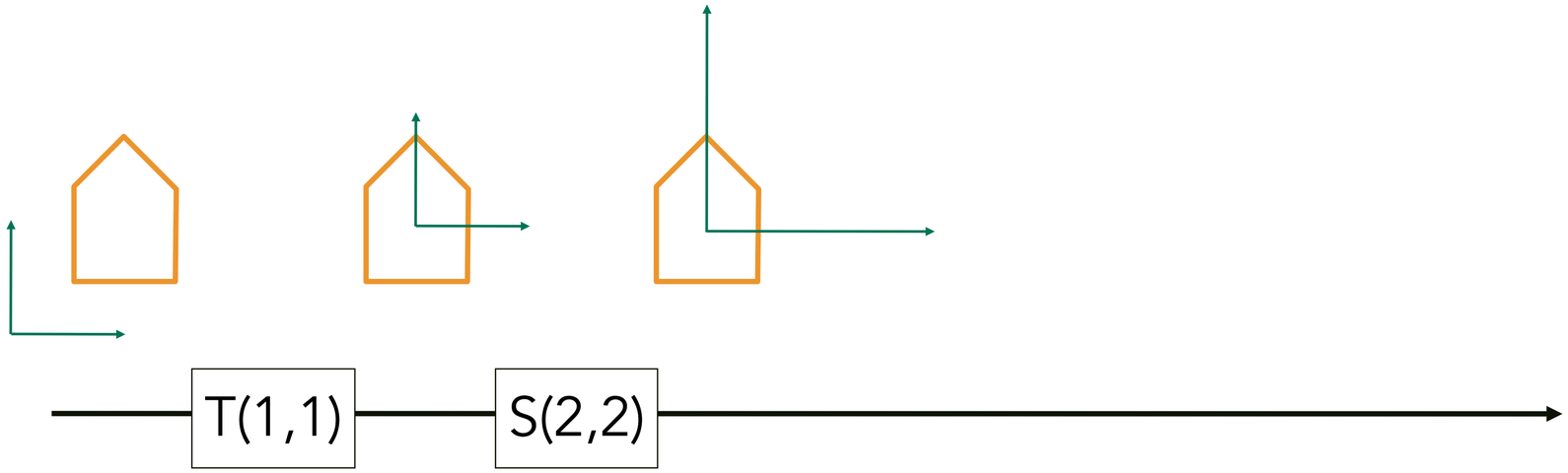
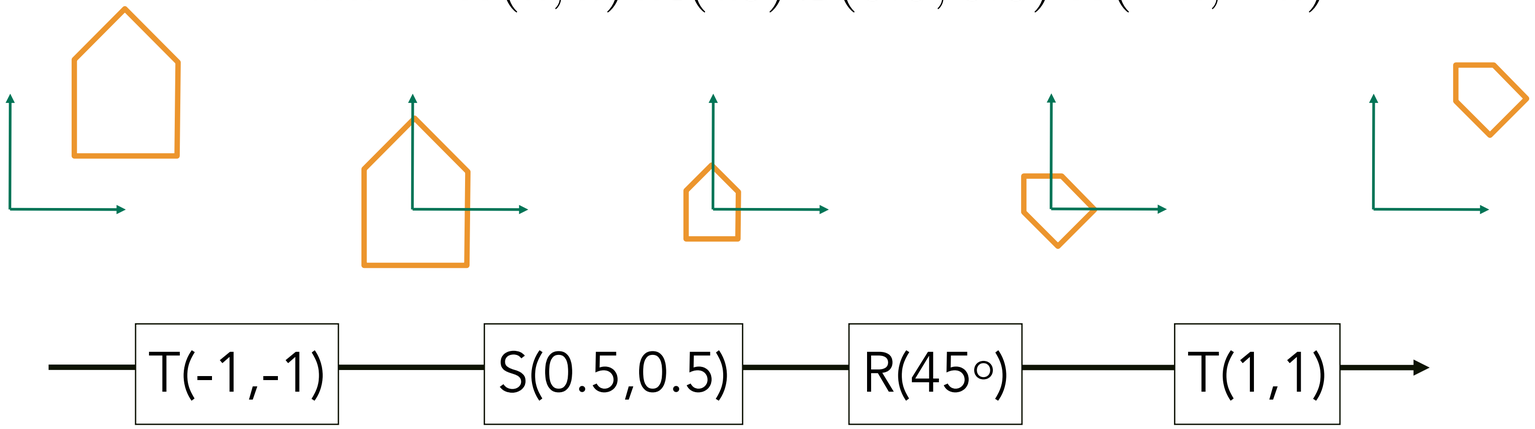


Image Copyright: Mark Pauly

Transform Object or Camera?

$$\mathbf{M} = \mathbf{T}(1,1)\,\mathbf{R}(45)\,\mathbf{S}(0.5,0.5)\,\mathbf{T}(-1,-1)$$



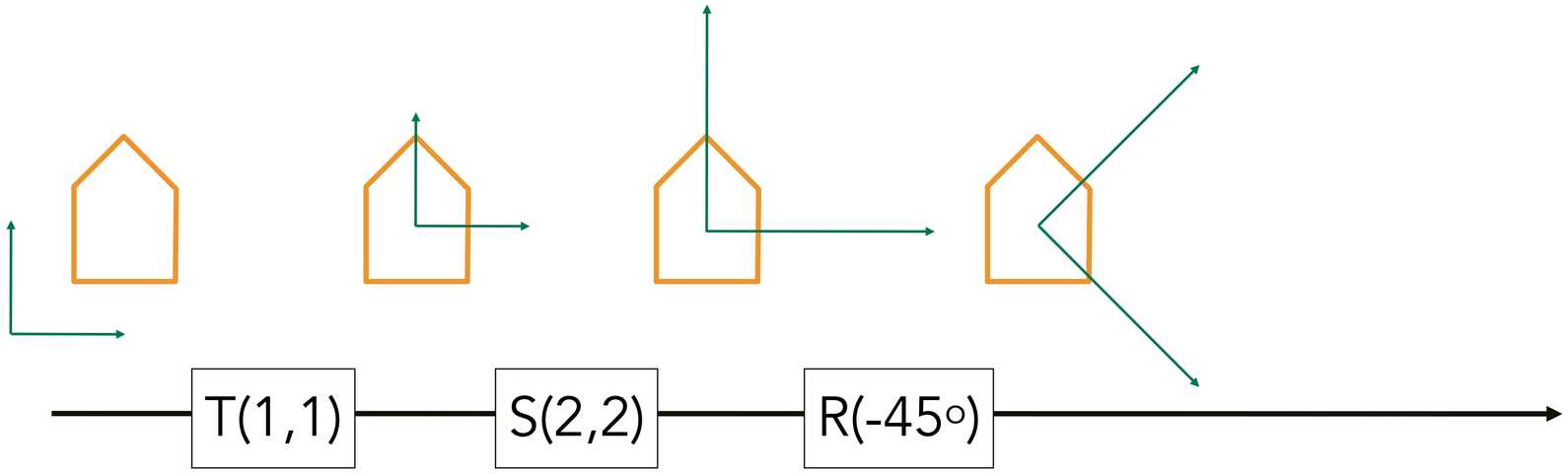
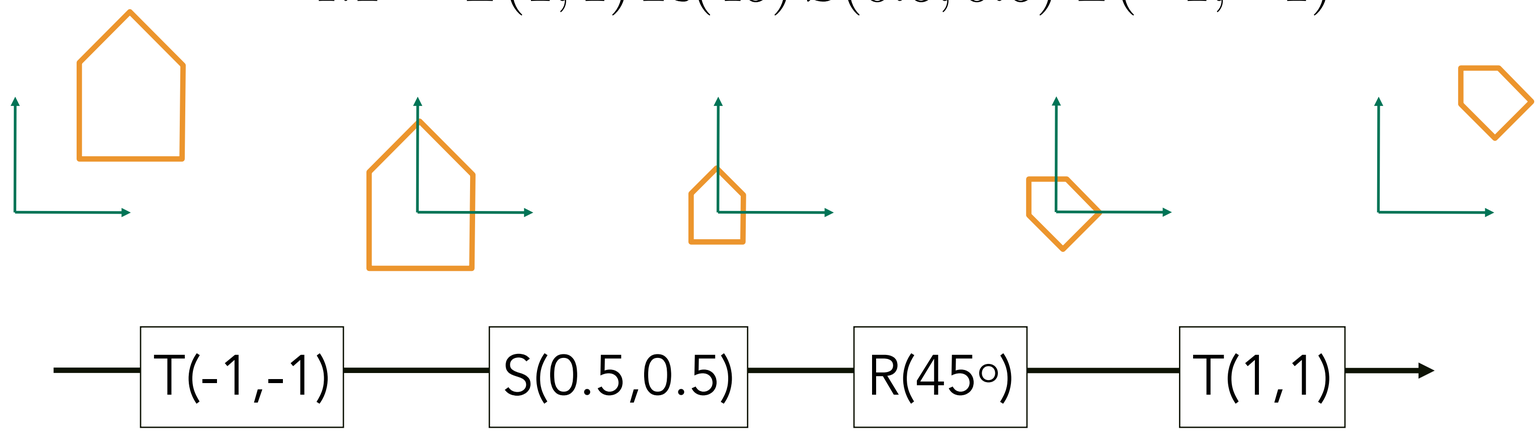
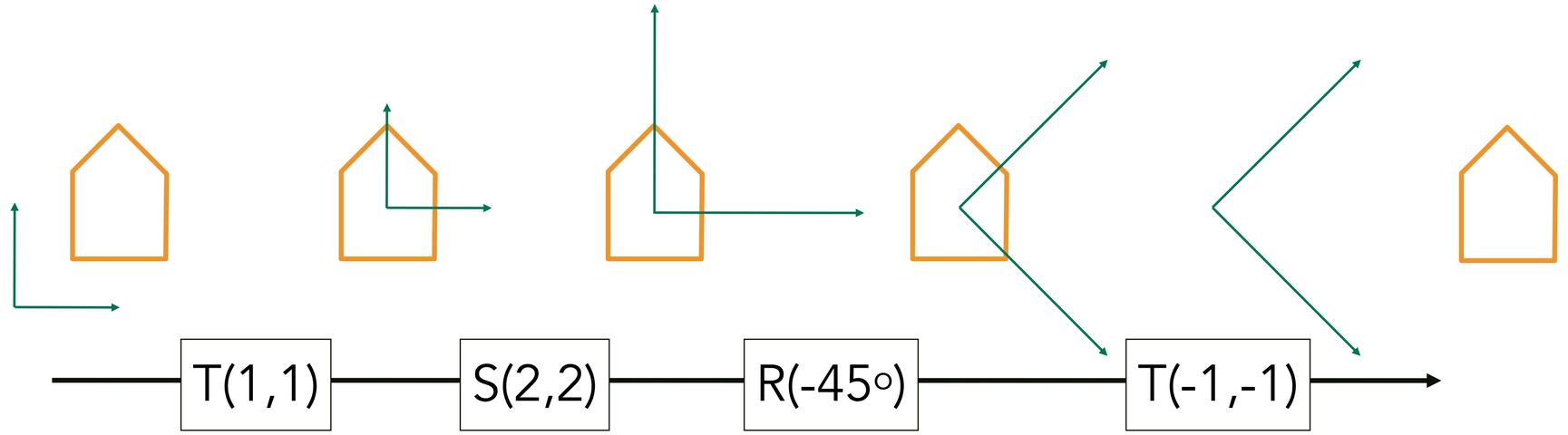


Image Copyright: Mark Pauly

Transform Object or Camera?

$$\mathbf{M} = \mathbf{T}(1,1)\,\mathbf{R}(45)\,\mathbf{S}(0.5,0.5)\,\mathbf{T}(-1,-1)$$



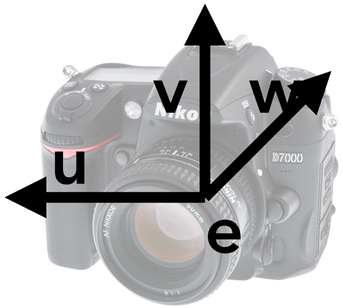


$$(\mathbf{T}(1,1)\,\mathbf{R}(45)\,\mathbf{S}(0.5,0.5)\,\mathbf{T}(-1,-1))^{-1} = \mathbf{T}(1,1)\,\mathbf{S}(2,2)\,\mathbf{R}(-45)\,\mathbf{T}(-1,-1) = \mathbf{M}^{-1}$$

Image Copyright: Mark Pauly

Transform Object or Camera?

A frame is a matrx wrt a world coordinate

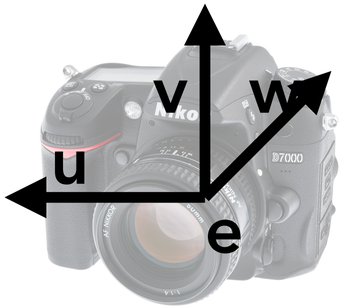


$$\mathbf{F} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{o} \end{bmatrix} = \begin{bmatrix} u_x & v_x & o_x \\ u_y & v_y & o_y \\ 0 & 0 & 1 \end{bmatrix}$$

University of Victoria | Computer Science

Transform Object or Camera?

A frame is a matrx wrt a world coordinate



$$\mathbf{F} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{o} \end{bmatrix} = \begin{bmatrix} u_x & v_x & o_x \\ u_y & v_y & o_y \\ 0 & 0 & 1 \end{bmatrix}$$

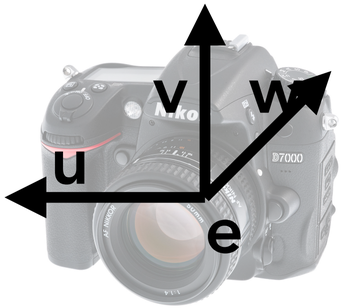
A point in the camera coordinate: $\mathbf{c} = (c_x, c_y, 1)$

The same point in the world coordinate: $\mathbf{Fc} = c_x\mathbf{u} + c_y\mathbf{v} + \mathbf{o}$

University of Victoria | Computer Science

Transform Object or Camera?

A frame is a matrx wrt a world coordinate



$$\mathbf{F} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{o} \end{bmatrix} = \begin{bmatrix} u_x & v_x & o_x \\ u_y & v_y & o_y \\ 0 & 0 & 1 \end{bmatrix}$$

A point in the camera coordinate: $\mathbf{c} = (c_x, c_y, 1)$

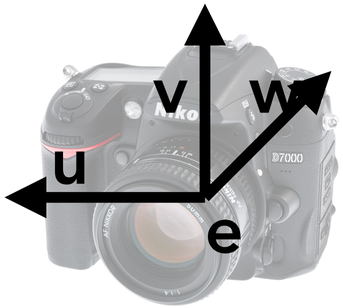
The same point in the world coordinate: $\mathbf{Fc} = c_x\mathbf{u} + c_y\mathbf{v} + \mathbf{o}$

A point **p** (world coordinate) in the camera coordinate: $\mathbf{c} = \mathbf{F}^{-1}\mathbf{p}$ **What a camera sees**

University of Victoria | Computer Science

Transform Object or Camera?

A frame is a matrx wrt a world coordinate



$$\mathbf{F} = \begin{bmatrix} \mathbf{u} & \mathbf{v} & \mathbf{o} \end{bmatrix} = \begin{bmatrix} u_x & v_x & o_x \\ u_y & v_y & o_y \\ 0 & 0 & 1 \end{bmatrix}$$

We have a point in the world coordinate: $\mathbf{x}$

What the camera sees: $\mathbf{c}_0 = \mathbf{F}^{-1}\mathbf{x}$

University of Victoria | Computer Science

Transform Object or Camera?

We have a point in the world coordinate: $\mathbf{X}$

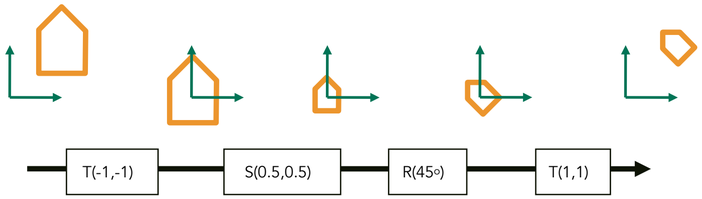
What the camera sees: $\mathbf{c}_0 = \mathbf{F}^{-1}\mathbf{x}$

University of Victoria | Computer Science

Transform Object or Camera?

We have a point in the world coordinate: $\mathbf{x}$

What the camera sees: $\mathbf{c}_0 = \mathbf{F}^{-1}\mathbf{x}$



Move the object by $\mathbf{M}$, camera fixed

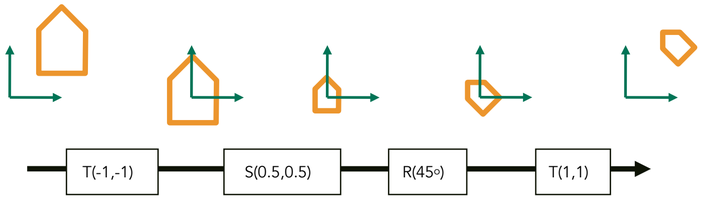
$$\mathbf{c}_1 = \mathbf{F}^{-1}(\mathbf{M}\mathbf{x}) = \mathbf{F}^{-1}\mathbf{M}\,\mathbf{x}$$

University of Victoria | Computer Science

Transform Object or Camera?

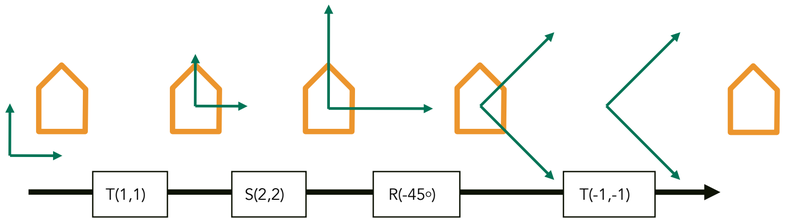
We have a point in the world coordinate: $\mathbf{X}$

What the camera sees: $\mathbf{c}_0 = \mathbf{F}^{-1}\mathbf{x}$



Move the object by **M**, camera fixed

$$\mathbf{c}_1 = \mathbf{F}^{-1}(\mathbf{M}\mathbf{x}) = \mathbf{F}^{-1}\mathbf{M}\,\mathbf{x}$$



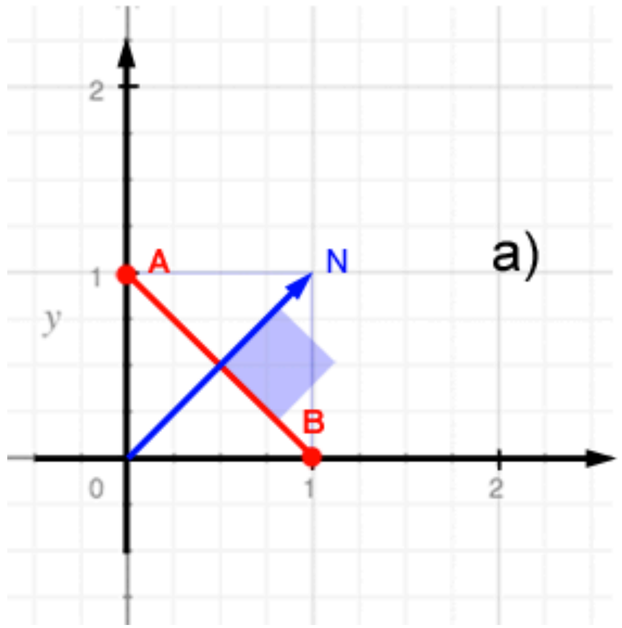
Move the camera by **M^{-1}**, object fixed

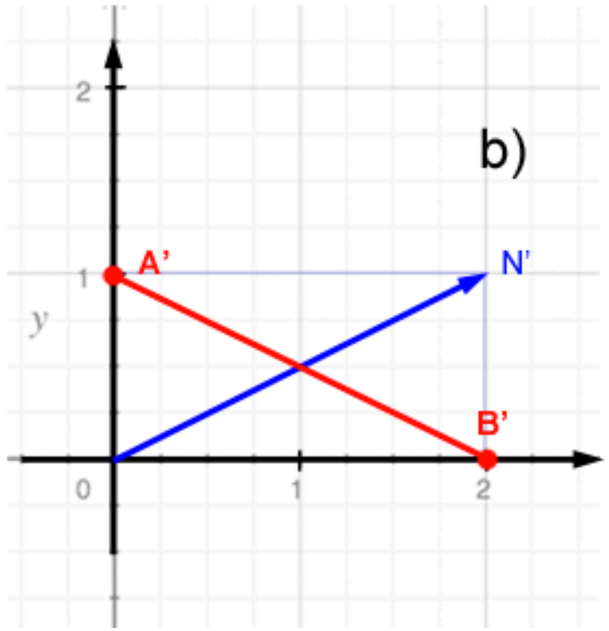
$$\mathbf{c}_2 = \left(\mathbf{M}^{-1}\mathbf{F}\right)^{-1}\mathbf{x} = \mathbf{F}^{-1}\left(\mathbf{M}^{-1}\right)^{-1}\mathbf{x} = \mathbf{F}^{-1}\mathbf{M}\,\mathbf{x}$$

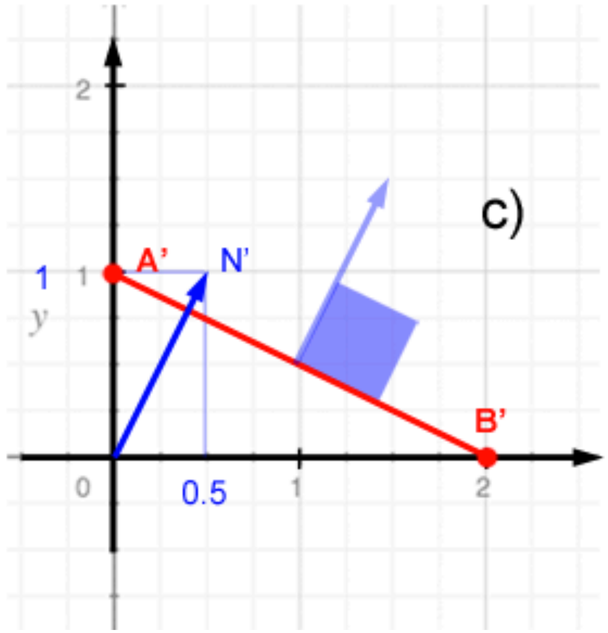
University of Victoria | Computer Science

A note on transforming normals

- If you transform a point **v** with a matrix M: **v'** = **Mv** …
- the transformed normal **n'** at the point **v** is **n' = M^{-T} n** (needs to be renormalized)



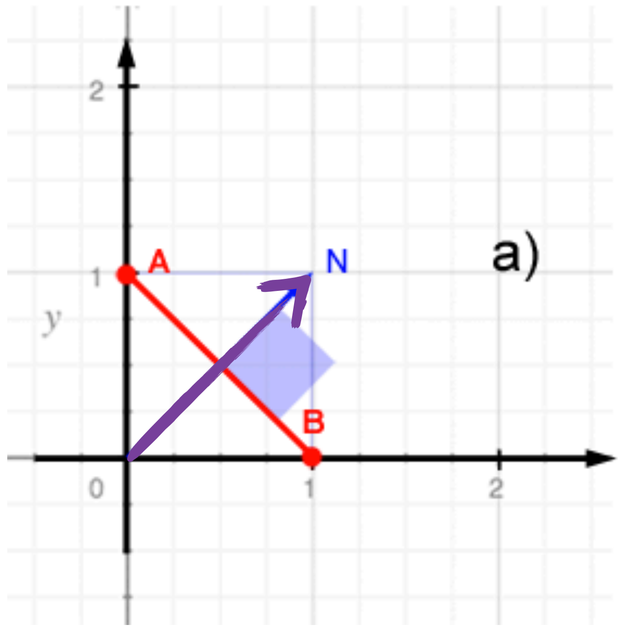


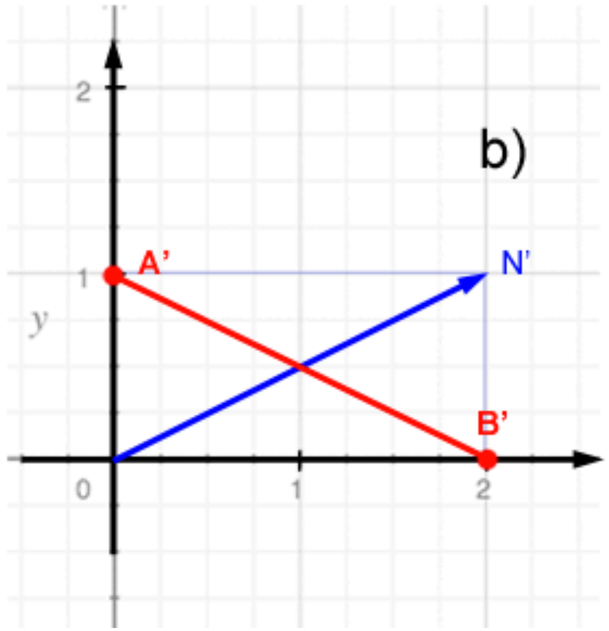


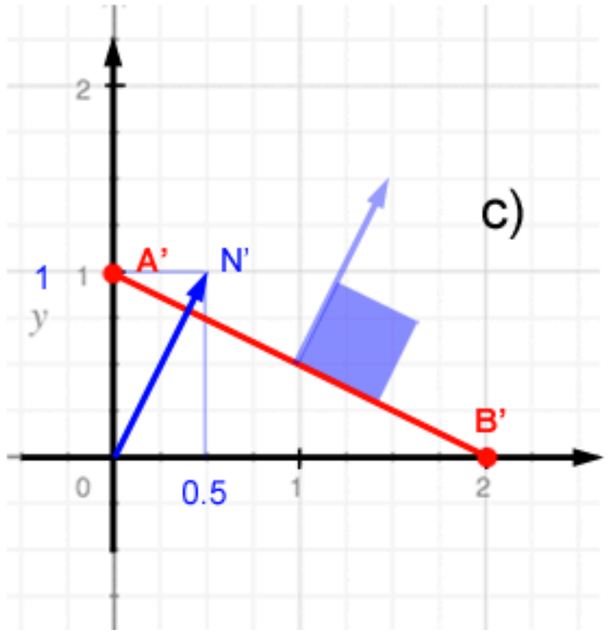
http://www.scratchapixel.com/lessons/mathematics-physics-for-computer-graphics/geometry/transforming-normals

A note on transforming normals

- If you transform a point **v** with a matrix M: **v'** = **Mv** …
- the transformed normal **n'** at the point **v** is **n' = M^{-T} n** (needs to be renormalized)





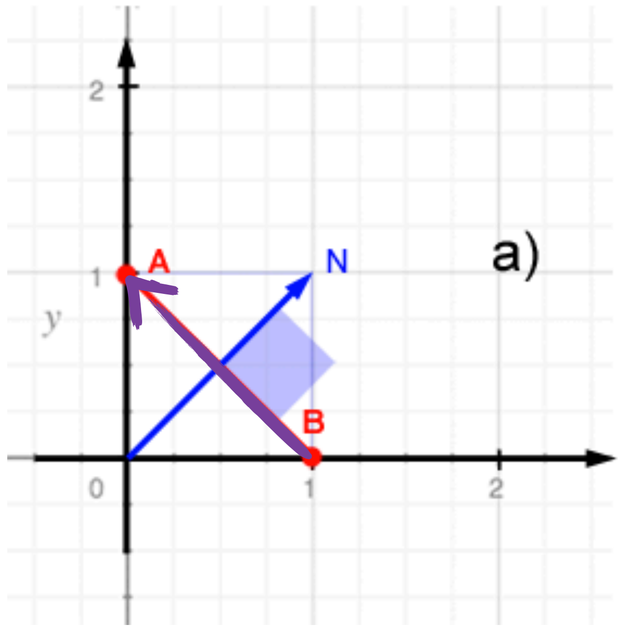


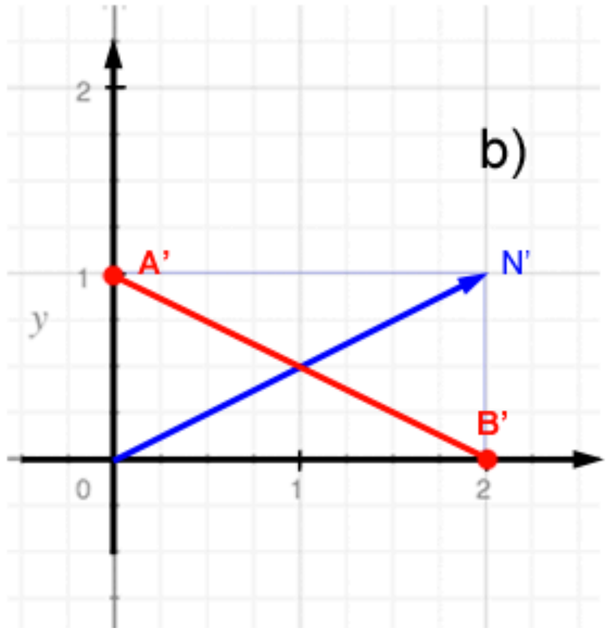
http://www.scratchapixel.com/lessons/mathematics-physics-for-computer-graphics/geometry/transforming-normals

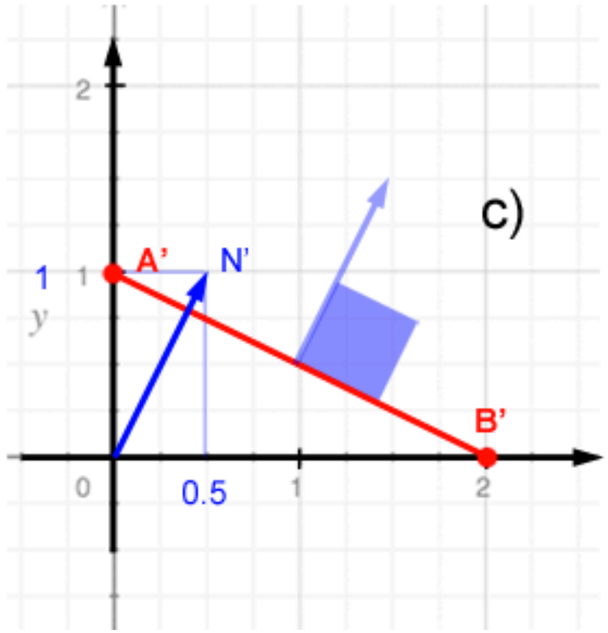
University of Victoria | Computer Science

A note on transforming normals

- If you transform a point **v** with a matrix M: **v'** = **Mv** …
- the transformed normal **n'** at the point **v** is **n' = M^{-T} n** (needs to be renormalized)





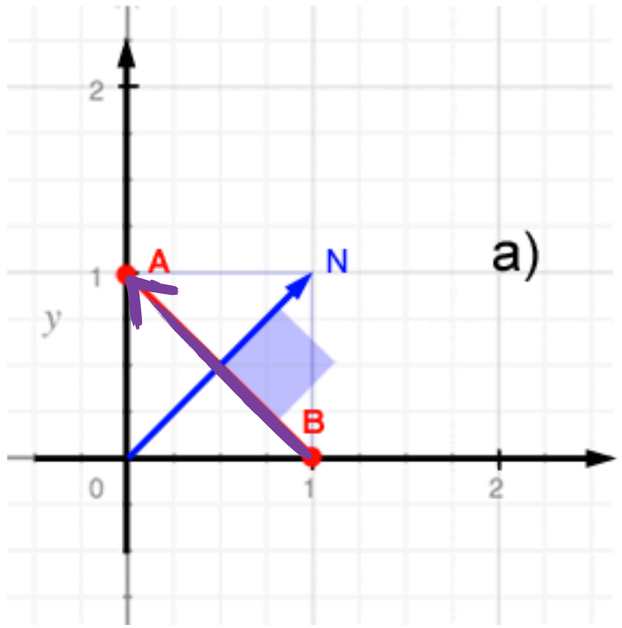


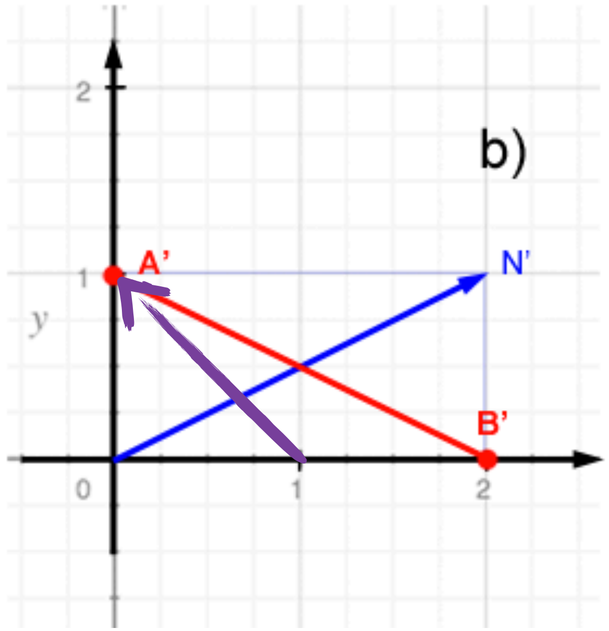
http://www.scratchapixel.com/lessons/mathematics-physics-for-computer-graphics/geometry/transforming-normals

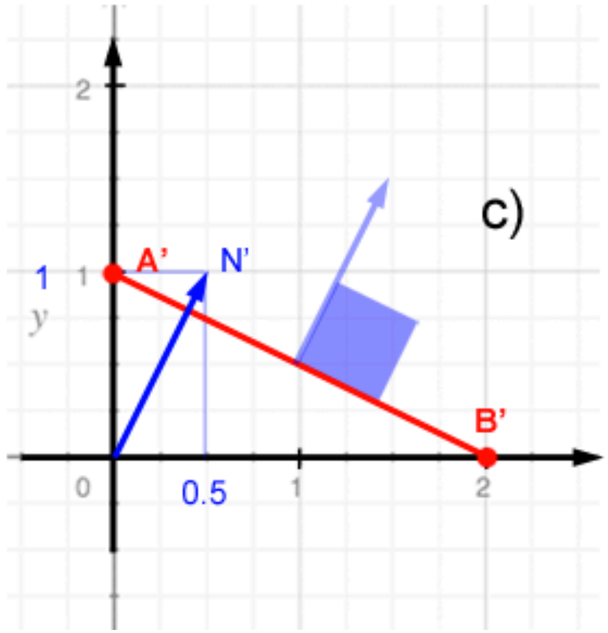
University of Victoria | Computer Science

A note on transforming normals

- If you transform a point **v** with a matrix M: **v'** = **Mv** …
- the transformed normal **n'** at the point **v** is **n' = M^{-T} n** (needs to be renormalized)



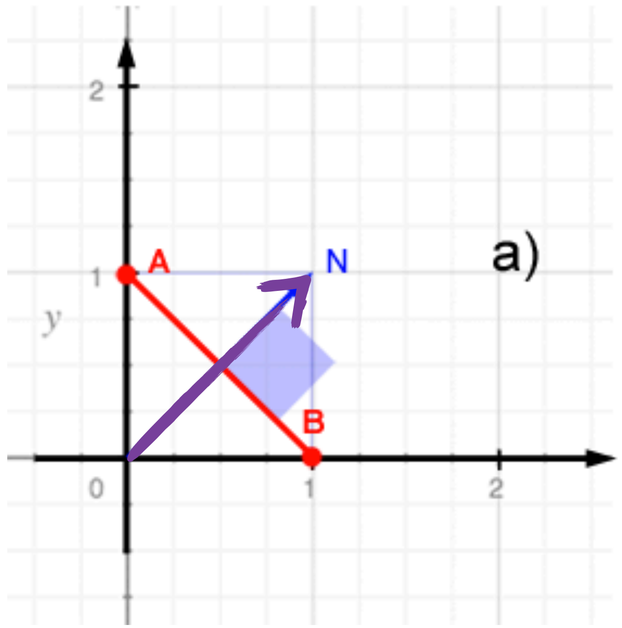


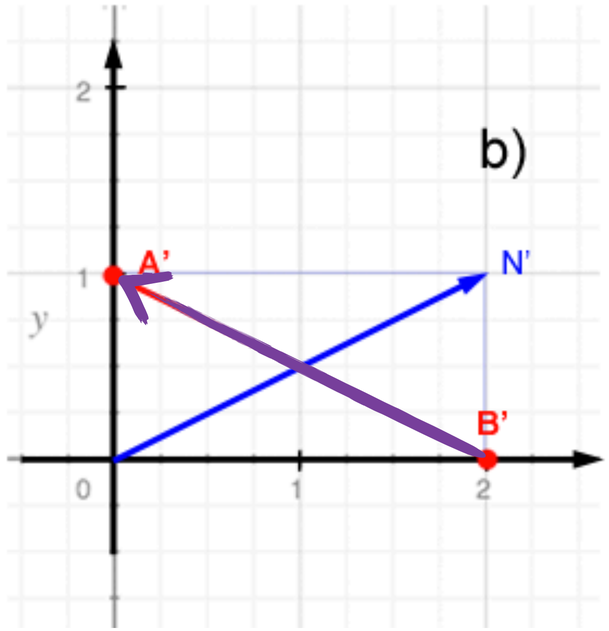


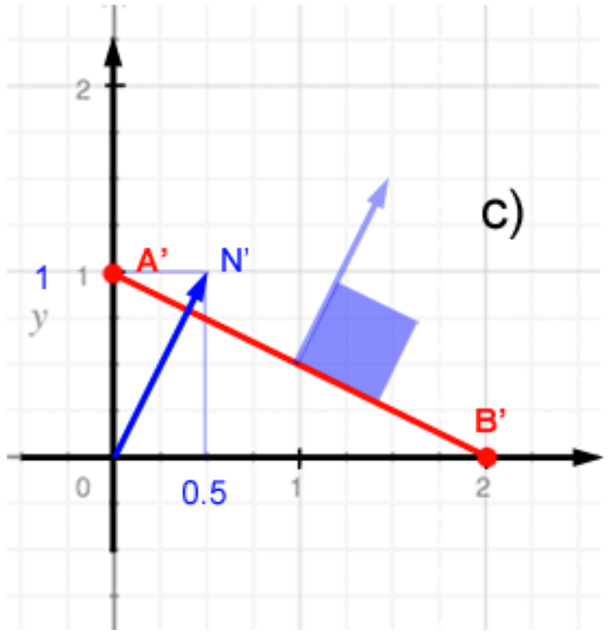
http://www.scratchapixel.com/lessons/mathematics-physics-for-computer-graphics/geometry/transforming-normals

A note on transforming normals

- If you transform a point **v** with a matrix M: **v'** = **Mv** …
- the transformed normal **n'** at the point **v** is **n' = M^{-T} n** (needs to be renormalized)





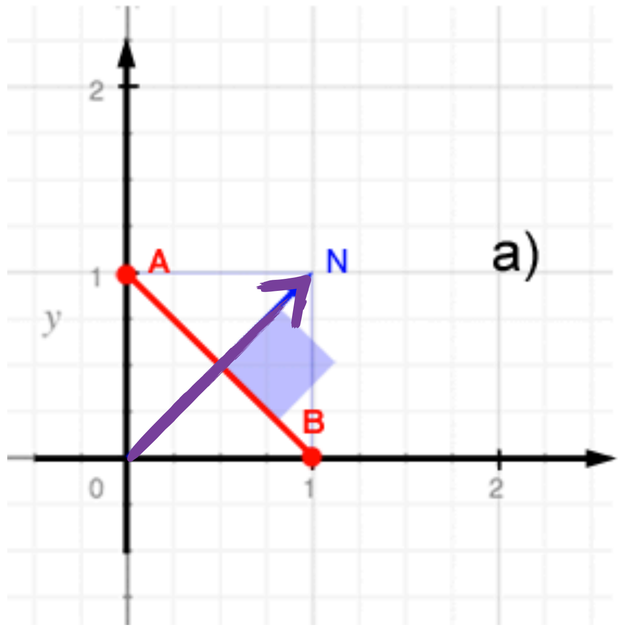


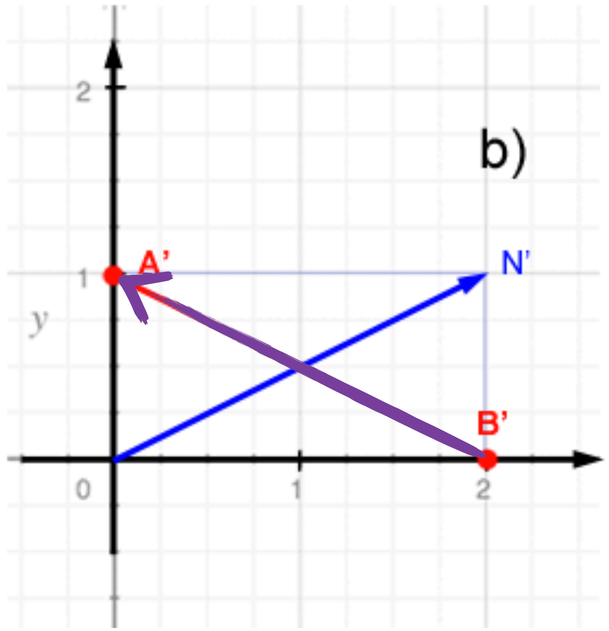
http://www.scratchapixel.com/lessons/mathematics-physics-for-computer-graphics/geometry/transforming-normals

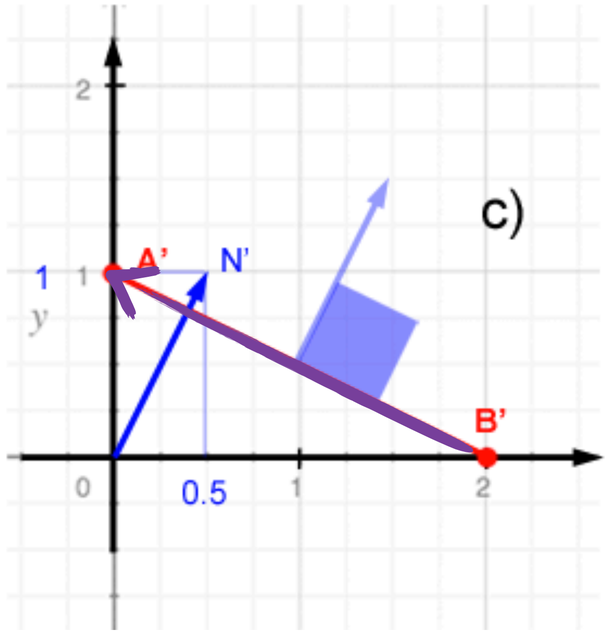
University of Victoria | Computer Science

A note on transforming normals

- If you transform a point **v** with a matrix M: **v'** = **Mv** …
- the transformed normal **n'** at the point **v** is **n' = M^{-T} n** (needs to be renormalized)





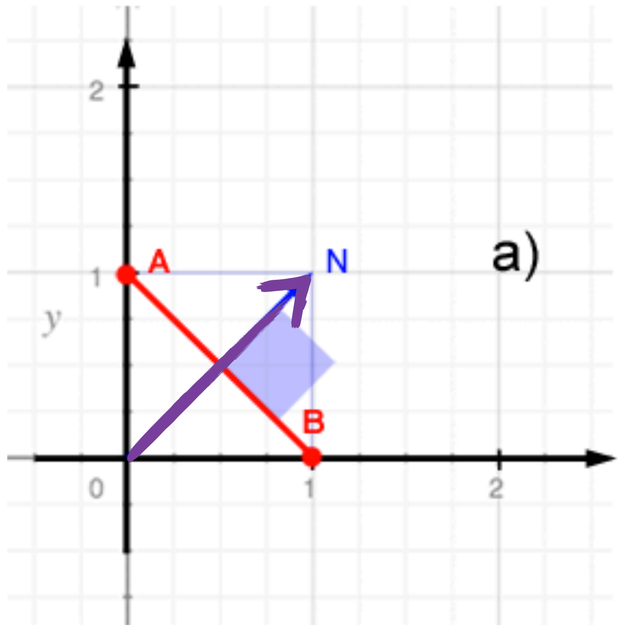


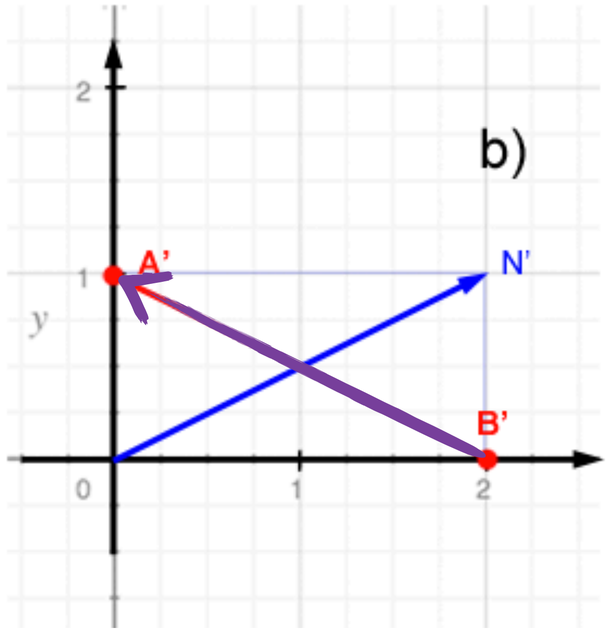
http://www.scratchapixel.com/lessons/mathematics-physics-for-computer-graphics/geometry/transforming-normals

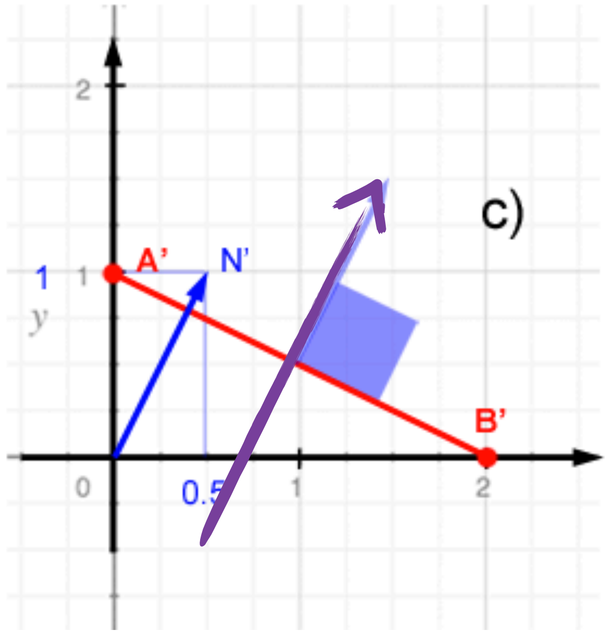
University of Victoria | Computer Science

A note on transforming normals

- If you transform a point **v** with a matrix M: **v'** = **Mv** …
- the transformed normal **n'** at the point **v** is **n' = M^{-T} n** (needs to be renormalized)







http://www.scratchapixel.com/lessons/mathematics-physics-for-computer-graphics/geometry/transforming-normals

References

Fundamentals of Computer Graphics, Fourth Edition
4th Edition **by Steve Marschner, Peter Shirley**

Chapter 6